MDPI

Article

Parallel Balanced Grey Wolf Optimizer: A Cooperative Parallel Approach for Large-Scale Optimization Problems

Glykeria Kyrou[1], Konstantinos G. Barkas [2], Vasileios Charilogis[3] and Ioannis G. Tsoulos[4]

[1] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece; g.kyrou@uoi.gr
[2] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece; kbarkas@uoi.gr
[3] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece; v.charilog@uoi.gr
[4] Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece; itsoulos@uoi.gr
* Correspondence: g.kyrou@uoi.gr

Abstract

In recent years, large-scale optimization problems have become increasingly common in various fields, such as machine learning and data analysis, generating increased references to both computational cost and accurate solutions. The Gray Wolf Optimizer (GWO) is an efficient collective intelligence algorithm, however its performance may be limited when high-permissive problems are allowed or in environments with strong multimodal landscapes. In this paper, we propose the Parallel Balanced Grey Wolf Optimizer (Parallel-BGWO), a parallel extension of GWO that aims to improve the balance between exploration and exploitation of the search space. The proposed algorithm divides the total population into multiple subpopulations, which cooperate through information exchange. This cooperation reduces the probability of premature convergence and enhances the global search. The parallel implementation exploits modern multi-core computing architectures, achieving a significant reduction in execution time, while at the same time maintaining or improving the quality of the final solutions. Experimental evaluations on high-dimensional benchmark functions show that ParallelBGWO exhibits faster convergence and reduced number of objective function evaluations compared to the classical version of GWO and other prominent methods. The results highlight ParallelBGWO as an efficient approach for demanding global optimization problems.

Keywords: Global optimization; GWO algorithm; Evolutionary techniques; Stochastic methods; Large-scale problems

1. Introduction

The basic goal of global optimization is to find the global minimum of a continuous multidimensional function and is defined as:

$$x^* = \arg\min_{x \in S} f(x) \tag{1}$$

with S:

$$S = [a_1, b_1] \times [a_2, b_2] \times \ldots [a_n, b_n]$$

with, a_i and b_i representing lower and upper bounds for each variable x_i.

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

In recent years, global optimization problems have become increasingly common in modern scientific fields, such as machine learning [1,2], big data analysis [3], communication systems [4], and energy management [5]. The increasing complexity of these applications leads to high-dimensional problems, making it particularly demanding to effectively explore the solution space of particular interest are Large-Scale Global Optimization (LSGO) problems [6] where the computational complexity increases significantly and classical deterministic methods often fail to find satisfactory solutions in reasonable time, either due to increased computational cost or due to trapping in local minima. The importance of LSGO problems was recognized internationally through competitions within the framework of the IEEE Congress on Evolutionary Computation (CEC) [7–9], which significantly strengthened research in this field.To address these challenges, there has been increasing interest in metaheuristic and stochastic algorithms that can efficiently approach the global optimum even in complex and high-dimensional environments. The Grey Wolf Optimizer (GWO) [10,11], is a widely used collective intelligence algorithm, inspired by the cooperative behavior of a wolf pack. Its simplicity of implementation and relatively fast convergence make it attractive for continuous optimization problems. However, in large-scale problems it exhibits limited exploration ability and an increased probability of premature convergence.

In this context, the Parallel Balanced Grey Wolf Optimizer (Parallel BGWO) is proposed, a parallel extension of GWO that aims to improve the balance between exploration and exploitation. The proposed algorithm divides the population into subpopulations that evolve simultaneously and exchange information, reducing the probability of being trapped at local extremes. In addition, a local optimization procedure is incorporated through the BFGS algorithm [12], while a termination rule [13], is implemented to effectively terminate the global optimization method. This modification avoids unnecessary calculations. Also, at the subpopulation level, different termination strategies (ANY, MAJORITY and ALL) are considered, which determine when the overall process is completed based on the convergence status of the individual clusters. The use of flexible termination criteria helps to avoid unnecessary calculations and improve the overall computational efficiency of the algorithm. The proposed algorithm was tested on real-world problems. As a representative real-world application, the training of Artificial Neural Networks (ANNs) can be formulated as a high-dimensional, non-convex optimization problem, where the objective is to determine the optimal set of model parameters by minimizing a predefined loss function. Due to the complexity of the search space and the presence of multiple local optima, conventional gradient-based methods may suffer from premature convergence. In this context, global optimization techniques provide a powerful alternative. For this reason, ANN training is employed in this work as an additional application to evaluate the effectiveness and robustness of the proposed Parallel BGWO algorithm.

This paper is divided as follows: in section2 presented the related literature and positions the proposed method within the current state of the art in section3 the BGWO algorithm, the proposed ParallelBGWO algorithm and a flowchart with a detailed description is presented, in section4 of the test functions used in the experiments as well as the related experiments are presented. In the 5section, there is a brief discussion of the results obtained from the experiments. In section 6some conclusions and directions for future improvements are discussed.

2. Literature review

Global optimization problems constitute a fundamental research topic across many scientific domains, as the need to determine optimal solutions arises in an increasing number of applications. The growing complexity of modern systems often leads to high-

dimensional problems, where classical deterministic methods frequently fail to provide reliable solutions. As a result, metaheuristic algorithms have emerged as an effective alternative for tackling complex optimization problems due to their ability to explore large search spaces and avoid local optima [14]. Among the earliest and most widely used metaheuristic approaches are evolutionary algorithms, which simulate the principles of biological evolution. Genetic Algorithms (GA) were among the first successful global search techniques, relying on mechanisms such as selection, crossover, and mutation to evolve populations of candidate solutions [14,15]. Later, Differential Evolution (DE) was introduced as a powerful optimization algorithm for continuous problems, offering a simple structure and stable convergence behavior. Due to its robustness, DE has been widely applied in engineering optimization and energy management systems [16,17]. Another important class of optimization methods is Swarm Intelligence algorithms, which are inspired by the collective behavior observed in natural systems. Particle Swarm Optimization (PSO) models the social interaction of particle swarms and has been successfully applied in global optimization and wireless communication problems [18,19]. Similarly, Ant Colony Optimization (ACO) simulates cooperative pathfinding behavior observed in ant colonies [20,21], while the Artificial Bee Colony (ABC) algorithm mimics the foraging behavior of honeybees and has been enhanced with learning mechanisms to improve its performance [22,23]. Other bio-inspired optimization methods include the Firefly Algorithm (FA) and the Bat Algorithm (BA), which exploit attraction mechanisms and echolocation strategies respectively to achieve an efficient balance between exploration and exploitation in the search process [24–27]. In recent years, research has increasingly focused on the development of metaheuristic algorithms capable of improving search efficiency. Algorithms such as the Whale Optimization Algorithm (WOA) [28,29], Harris Hawks Optimization (HHO) [30,31], Aquila Optimizer (AO) [32,33], Arithmetic Optimization Algorithm (AOA) [34,35], Magnificent Frigatebird Optimization Algorithm (MFO) [36] and Salp Swarm Algorithm (SSA) [37,38] attempt to enhance the balance between global exploration and local exploitation through adaptive search strategies. Among swarm-based optimization algorithms, the Grey Wolf Optimizer (GWO) has attracted considerable attention due to its simplicity and effectiveness in solving continuous optimization problems. Inspired by the leadership hierarchy and cooperative hunting behavior of grey wolves, GWO models the interaction between alpha, beta and delta wolves to guide the search process toward promising regions of the solution space [10,11]. Despite its effectiveness, the original GWO algorithm may suffer from premature convergence and reduced exploration ability when applied to large-scale or complex optimization problems.

To address these limitations, several improved variants of the GWO algorithm have been proposed in recent years. Chen et al. introduced the Balanced Grey Wolf Optimizer (BGWO), which aims to maintain a better balance between exploration and exploitation during the optimization process [41]. The proposed method was applied to optimize the weights of Backpropagation Neural Networks, improving the convergence behavior and predictive performance of the neural model. Similarly, Adhikary and Acharyya proposed the Randomized Balanced Grey Wolf Optimizer (RBGWO), a stochastic extension of BGWO that incorporates randomized mechanisms to enhance the diversity of the search process [42]. This approach improves the ability of the algorithm to escape local optima and achieve better performance in real-world optimization problems. Furthermore, Wang et al. developed an Adaptively Balanced Grey Wolf Optimizer (ABGWO), which dynamically adjusts the balance between exploration and exploitation throughout the search process [43]. Their approach was applied to feature selection in high-dimensional classification problems, demonstrating improved classification accuracy while reducing data dimensionality. At the same time, recent studies emphasize the importance of parallel and collaborative

metaheuristic strategies, where the population is divided into multiple subpopulations that explore different regions of the search space simultaneously. The exchange of information between these subpopulations helps improve diversity, reduce premature convergence, and accelerate the optimization process. Such collaborative approaches have been successfully demonstrated in modern parallel implementations of algorithms such as Smell Agent Optimization (SAO) [39,40]. Overall, the evolution of metaheuristic optimization methods reflects a clear transition from classical evolutionary techniques toward hybrid, adaptive, and parallel optimization frameworks, capable of effectively addressing the challenges posed by complex large-scale optimization problems in modern applications.

To address these limitations, parallel and cooperative optimization strategies have recently attracted increasing attention. By dividing the population into multiple interacting subpopulations, parallel approaches allow different regions of the search space to be explored simultaneously, improving diversity and reducing the probability of stagnation in local optima. Furthermore, hybridization with local optimization techniques can significantly improve convergence speed and solution accuracy. Motivated by these observations, this work proposes a Parallel Balanced Grey Wolf Optimizer (Parallel BGWO) designed to enhance the search efficiency of the classical GWO framework. The proposed method introduces a subpopulation-based parallel structure, where multiple groups of candidate solutions evolve simultaneously while exchanging information to maintain diversity. In addition, a local search mechanism based on the BFGS algorithm is incorporated to refine promising solutions and accelerate convergence. Also an adaptive termination rule is employed to avoid unnecessary computations when the algorithm approaches convergence. At the same time, flexible termination strategies (ANY, MAJORITY, ALL) are applied at the subpopulation level, which determine when the overall process is completed based on the convergence status of the individual clusters. The effectiveness of the proposed approach is evaluated through extensive experiments on benchmark optimization problems and real-world applications.

3. Materials and Methods

3.1. *The BGWO algorithm*

The Balanced Grey Wolf Optimizer (BGWO) is an improved version of the Grey Wolf Optimizer (GWO), aiming to improve the balance between global exploration and local exploitation. In BGWO, each search agent represents a wolf, whose position corresponds to a candidate solution in the D-dimensional search space. First, a population of N wolves is randomly generated within the allowable limits of the problem. Then, the objective function value is calculated for each wolf and a fitness ranking is performed. The three best solutions are defined as wolves a, b and d and constitute the leading trio of the population. The remaining wolves update their positions guided by the three leaders, according to the prey encirclement mechanism that characterizes GWO. In each iteration of the algorithm, a nonlinear convergence factor a is calculated, which controls the dynamics of the search. In contrast to the classic GWO, where the factor decreases linearly, in BGWO a nonlinear decay is used, which allows for more intense exploration in the initial iterations and a gradual increase in exploitation in the final phases of the process. For each wolf x_i, random coefficients are generated and used to calculate the control vectors and distances from the three leaders. The distances are calculated per dimension, taking into account the relative position of each wolf with respect to x_α, x_β *and* x_δ. In the BGWO algorithm there is a position update mechanism where independent stochastic scaling factors R_1, R_2, R_3, in the interval $[0.5, 1.5]$, are introduced. These factors randomly strengthen or weaken the influence of each leader, reducing excessive dependence on the current optimal solutions and enhancing population differentiation. For each wolf, three intermediate positions are

Algorithm 1 BGWO algorithm

INPUT
- f: objective function
- N: population size (number of wolves)
- D: problem dimension
- T: maximum number of iterations
- k: convergence coefficient // BGWO parameter
- N_t: termination criterion

$p_l \in [0, 1]$ as the local search rate.
OUTPUT
-x_a: best solution (alpha wolf)
INITIALIZATION
- **Set** $iter \leftarrow 0$ as the iteration counter
- **Initialize** wolves $x_i \in R^D, i \in \{1..N\}$
- **Compute** fitness $F_i = f(x_i)$ for each wolf
- **Identify** the best three wolves: $x_a(best), x_b, x_c$

main pseudocode
while stopping criterion is not met do
  Compute nonlinear convergence factor: $a \leftarrow 2\exp^{\frac{k}{ln\frac{t-1}{T}}}$
  for each wolf $i, i \in \{1..N\}$ **do**
    Draw $r_1, r_2, r_3, r_4, r_5, r_6 \backsim U(0, 1)$
    Compute coefficient vectors: $C_a = 2 \cdot r_1, C_b = 2 \cdot r_2, C_c = 2 \cdot r_3$
    Compute A-vectors:$A_a = a \cdot (2 \cdot r_4 - 1)$, $A_b = a \cdot (2 \cdot r_5 - 1)$, $A_c = a \cdot (2 \cdot r_6 - 1)$
    Compute dinstances to leaders: $D_a = |C_a \cdot x_a - x_i|, D_b = |C_b \cdot x_b - x_i|, D_c = |C_c \cdot x_c - x_i|$
    Draw stochastic scaling factors $R1, R2, R3, \backsim U(0.5, 1.5)$
    Compute stochastic guided positions(BGWO update):
    $x_a = R_1 \cdot (x_a - A_a \cdot D_a)$,$x_b = R_2 \cdot (x_b - A_b \cdot D_b)$,$x_c = R_3 \cdot (x_c - A_c \cdot D_c)$
    Update wolf position: $x_i \leftarrow \frac{x_a + x_b + x_c}{3}$
    Draw $r \backsim U(0, 1)$
    if $r \leq p_l$ **then**
      Apply local search$x_i \leftarrow LS(x_i)$. In this procedure the local optimization method of Powell [12] was used.
    end if
  endfor
  Set $iter = iter + 1$
  if $iter \geq T$ **then** terminate
    Evaluate $F_i \leftarrow f(x_i)$ for all wolves
    Update x_a,x_b,x_c wolves based on best fitness values
    Compute $\delta^{(k)} = |\sum_{i=1}^{N} |f_i^{(iter)}| - \sum_{i=1}^{N} |f_i^{(iter-1)}|$. This termination method was introduced in [13].
    if $\delta^{(k)} \leq \epsilon$ **for** $N_t$iterations **then** terminate.
      Update x_a,x_b,x_c wolves based on best fitness values
end while // Goto termination check
Returnx_a

calculated, each of which is guided by one of the leaders. The final updated position is the average of these three stochastic scaled positions.

This procedure improves the ability to escape from local minima and makes the algorithm more robust to problems with high multimodality. After all wolves have been updated, the objective function is recalculated and the leading trio is redefined based on the new ranking. The iterative process continues until some termination criterion is met. The steps of BGWO algorithm are presented in the following Algorithm.

3.2. *The parallel algorithm of BGWO*

The parallel implementation of the Balanced Grey Wolf Optimizer (BGWO) is based on dividing the population into N subpopulations, which are executed independently on different processing units. First, the parameters are initialized and the termination strategy T_S is selected. In each iteration, it is calculated how many subpopulations have satisfied the termination criterion. This information is compared with a threshold Q, which is determined by the T_S strategy as follows: ANY where the algorithm proceeds when at least one subpopulation terminates, MAJORITY where at least the majority of the subpopulations are required to have terminated and ALL where all subpopulations must have terminated. Then, each subpopulation evolves in parallel by performing the basic steps of BGWO (evaluation, leader selection, position update). At intervals, a propagation mechanism is applied to exchange good solutions between the subpopulations. The iteration counter is incremented and the overall termination criterion is checked. If it is not satisfied, the process is repeated. Upon termination, a local search is applied to the best

solution x_{best} and the final result is returned. The steps of Parallel BGWO algorithm are presented in the following Algorithm.

Algorithm 2 The parallel algorithm of BGWO

INPUT
- f: objective function
- N: number of subpopulations
- N_M: propagation method
- N_R: propagation rate
- N_P: number of agents for propagation
- N_I: termination criterion
- T_S: termination Strategy, $T_S \in$ {ANY, MAJORITY, ALL}

OUTPUT
-X_{best}: global best solution over all subpopulations

INITIALIZATION
- **Set** $iter \leftarrow 1$
- **Select** termination strategy T_S

MAIN LOOP

```
Compute the number of terminated subpopulations.
If T_S ← ANY then
    Q ← 1
Elseif T_S ← MAJORITY then
    Q ← N/2
Else T_S ← ALL then
    Q ← N
Endif
For k ∈ {1..N} do in Parallel
    Execute for each subpopulation k all the instructions in the inner part of the while of the basic algorithm
EndFor
If iter mod N_R ← 0, apply the propagation scheme N_M with N_P agents to the subpopulations.
    Set iter ← iter + 1
Endif
If According to parameter N_I its termination rule does not apply, go to Main Step.
    Apply local search procedure to x_best.
Endif
```

Furthermore, the steps of the proposed method are also graphically shown in Figure 1.

Figure 2 illustrates the creation of parallel processes and the information propagation mechanism in the parallel BGWO framework. Each thread $1, 2, \ldots, N$ corresponds to an independent processing unit (PU), which manages a subpopulation. The subpopulations evolve simultaneously, exploring different regions of the search space through the basic mechanisms of BGWO. Parallel execution allows for independent search for solutions, while at the same time supporting cooperation through propagation mechanisms. During this phase, selected high-quality solutions are transferred between subpopulations according to predefined exchange strategies. When a subpopulation finds a particularly good solution, its propagation to other subpopulations can accelerate the overall convergence. Overall, the combination of parallel exploration and controlled information exchange makes parallel BGWO more efficient, allowing a better balance between exploration and exploitation.

3.3. *The propagation mechanism*

During the propagation mechanism, the best values identified by each subpopulation are shared with the others, replacing their worst values. The goal of this process is to ensure that the subpopulations exchange their optimal findings, thereby enhancing the overall evolutionary process. The following four scenarios describe how propagation can take place:

1. One-to-One(1to1): In this case, a randomly selected subpopulation sends its bests values to another subpopulation, also chosen at random.
2. One-to-All(1toN): Here, a random subpopulation shares its bests values with all other subpopulations.
3. All-to-One(Nto1): All subpopulations send their bests values to a single subpopulation, which is chosen at random.
4. All-to-All(NtoN): Each subpopulation communicates its bests values to every other subpopulation.

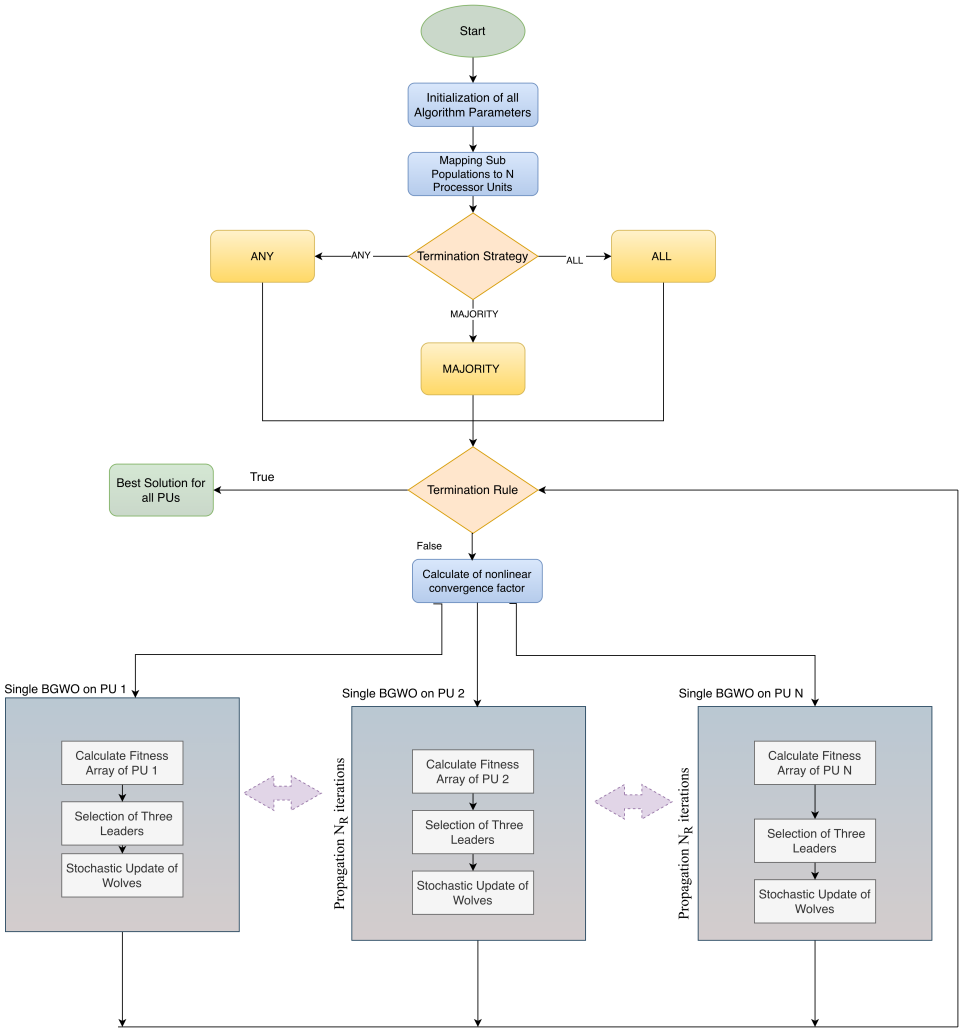

Figure 1. Flowchart of parallel BGWO

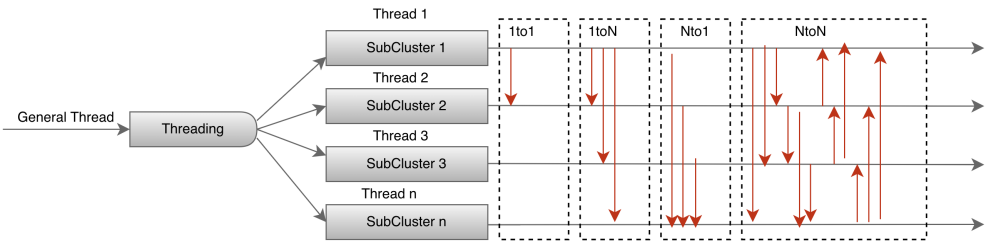

Figure 2. Thread creation diagram and propagation methods.

Table 1. Benchmark test functions used in experimental study.

NAME	FORMULA	DIM	G_{min}
ATTRACTIVE SECTOR	$f(x) = \left(\sum_{i=1}^{n}(s_i x_i)^2\right)^{0.9}$	2	0
BUCHE RASTRIGIN	$f(\mathbf{x}) = \sum_{i=1}^{n}[z_i \cdot (1 + 0.1 \cdot \sin(10\pi z_i))]$	n	0
DIFFERENT POWERS	$f(\mathbf{x}) = \sqrt{\sum_{i=1}^{n} \lvert x_i \rvert^{2+4\frac{i-1}{n-1}}}$	n	0
DISCUS	$f(x) = 10^6 x_1^2 + \sum_{i=2}^{n} x_i^2$	n	0
ELLIPSOIDAL	$f(x) = \sum_{i=1}^{n}\left(10^6\right)^{\frac{i-1}{n-1}} x_i^2$	n	0
GALLAGHER101	$f(\mathbf{x}) = \max_{i=1}^{101}\left[h_i - w_i\sqrt{\sum_{j=1}^{n}(x_j - c_{ij})^2}\right] \; min: 100 + 1$	n	0
GALLAGHER21	$f(\mathbf{x}) = \max_{i=1}^{21}\left[h_i - w_i\sqrt{\sum_{j=1}^{n}(x_j - c_{ij})^2}\right] \; min: 10 + 10 + 1$	n	0
GRIEWANK ROSENBROCK	$f(\mathbf{x}) = \underbrace{\left(\frac{\lVert\mathbf{x}\rVert^2}{4000} - \prod_{i=1}^{n}\cos\left(\frac{x_i}{\sqrt{i}}\right) + 1\right)}_{\text{Griewank}} \cdot \underbrace{\left(\frac{1}{10}\sum_{i=1}^{n-1}\left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]\right)}_{\text{Rosenbrock}}$	n	0
GRIEWANK	$f(x) = 1 + \frac{1}{200}\sum_{i=1}^{n} x_i^2 - \prod_{i=1}^{n}\frac{\cos(x_i)}{\sqrt{(i)}}$	n	0
RARSTIGIN	$f(\mathbf{x}) = An + \sum_{i=1}^{n}\left[x_i^2 - A\cos(2\pi x_i)\right] \; A = 10$	n	0
ROSENBROCK	$f(x) = \sum_{i=1}^{n-1}\left(100\left(x_{i+1} - x_i^2\right)^2 + (x_i - 1)^2\right), \quad -30 \leq x_i \leq 30$	n	0
SHARP RIDGE	$f(\mathbf{x}) = x_1^2 + \alpha\sum_{i=2}^{n} x_i^2, \; a > 1$	n	0
STEP ELLIPSOIDAL	$f(\mathbf{x}) = \sum_{i=1}^{n}\lfloor x_i + 0.5\rfloor^2 + \alpha\sum_{i=1}^{n}\left(10^6 \cdot \frac{i-1}{n-1}\right)x_i^2, \; a = 1$	n	0
ZAKHAROV	$f(\mathbf{x}) = \sum_{i=1}^{n} x_i^2 + \left(\sum_{i=1}^{n}\frac{i}{2}x_i\right)^2 + \left(\sum_{i=1}^{n}\frac{i}{2}x_i\right)^4$	n	0

This mechanism facilitates information exchange among subpopulations, increasing the likelihood of discovering optimal solutions through collaboration and communication.

4. Experiments

This section begins with a description of the functions that will be used in the experiments and then presents in detail the experiments that were performed, in which the parameters available in the proposed algorithm were studied, in order to study their reliability and adequacy.

4.1. Test Functions

A variety of benchmark test functions were used in the conducted experiments. These functions have been widely adopted in previous studies [44–47]. In the present work, the test functions are evaluated using dimensionalities ranging from 25 to 150, where the constant n denotes the dimension of the objective function. The benchmark test functions used in the experimental study are summarized in Table 1, including their mathematical formulation, dimensionality, and global optimum values.

4.2. Experimental results

A series of experiments were carried out for the previously mentioned functions and these experiments were executed on an AMD RYZEN 5950X with 128GB RAM. The operating system of the running machine was Debian Linux. Each experiment was conducted 30 times, with different random numbers each time, and the averages were recorded. The software used in the experiments was coded in C++ using the freely available optimization environment of OPTIMUS [48], which can be downloaded from https://github.com/itsoulos/OPTIMUS. All comparative methods were independently implemented and evaluated within the same experimental framework and were not directly adopted from the corresponding literature. All algorithms were implemented in C++ and executed under identical hardware and software conditions to ensure a fair and consistent comparison. Specifically, the number of agents (population size) was set to 200 for all algorithms, the maximum number of iterations was fixed at 200, and the local search rate was uniformly set to 0.05. These shared settings ensure that performance differences arise from algorithmic design rather than differences in experimental configuration. Algorithm-

specific parameters were selected according to standard practices reported in the literature and were kept constant throughout all experiments. No problem-specific parameter tuning was applied in order to avoid bias and to maintain methodological consistency. The values used for each parameter are shown in Table 2.

Table 2. The values for the experimental parameters.

PARAMETER	VALUE	EXPLANATION
w	200	Number of wolves
$iter_{max}$	200	Maximum number of iterations
N	1,3,10,20,30	Number of subpopulations
SR	$\delta_k^{(iter)} = \left\lvert f_{k,min}^{(iter)} - f_{k,min}^{(iter-1)} \right\rvert$	Stopping rule: Similarity
P_L	0.05 (5%) etc.	Local search rate
N_M	1to1, 1toN, Nto1, NtoN	Propagation method
N_R	1,2,3,4...$\leq iter_{max}$	Number of iterations for propagation
N_P	1,2,3,4...$\leq m$	Number of agents for propagation
N_T	1,2,3,4...$\leq N$	Number of subpopulations required for termination

4.3. Impact of the Number of Clusters

The 3 presents a comparative analysis of the number of objective function evaluations for a set of benchmark functions, in dimensions 25, 50, 100 and 150. The comparison is made in terms of the number of clusters used in the parallel or cooperative execution of the algorithm: 1, 3, 10, 20 and 30 clusters. The results show that when the population is divided into more clusters the number of required function calls is reduced, which means better computational efficiency. For example, in Büche Rastrigin_100 and Different Powers_100, where 1 cluster shows a higher computational cost of 17,769 and 23,887 respectively, in Büche Rastrigin_150 and Different Powers_150, it is 17,275 and 27,185 respectively. Therefore, the division of the population into subpopulations enhances diversity and promotes the parallel exploration of different regions of the search space, reducing premature convergence. It is also observed that 10 and 20 clusters tend to achieve the lowest or comparably low function calls values, for example for Büche Rastrigin_150 it is 6828 and 6837 respectively and for Different Powers_150 it is 12003 and 12858 respectively, which indicates a more effective balance between exploration and exploitation. On the contrary, further increase to 30 clusters does not lead to improvement and in several cases an increase in cost is observed. The total number of function calls decreases from 384,022 (1 cluster) to 363,819 (3 clusters), while a decrease is observed for 10–20 clusters (173,392-174,401). On the contrary, in the case of 30 clusters, the total cost increases (228,652). In conclusion, the cooperative structure of multiple subpopulations can significantly reduce the computational cost.

Table 3. Experimental results using different number of clusters.

FUNCTION	1 CLUSTER	3 CLUSTER	10 CLUSTER	20 CLUSTER	30 CLUSTER
ATTRACTIVE SECTOR_25	835	1292	1171	1170	1106
ATTRACTIVE SECTOR_50	1715	1851	1549	1534	1481
ATTRACTIVE SECTOR_100	2319	2261	1818	1809	1833
ATTRACTIVE SECTOR_150	2123	2113	1754	1754	1722
BUCHE RASTRIGIN_25	3077	4451	1606	1578	1959
BUCHE RASTRIGIN_50	10475	9890	4514	4420(0.97)	5220(0.90)
BUCHE RASTRIGIN_100	17769	15782	7302	7006(0.97)	10690(0.90)
BUCHE RASTRIGIN_150	17275	15319	6828	6837(0.97)	10623(0.97)
DISCUS_25	914	1391	1221	1153	1116
DISCUS_50	1901	2013	1513	1549	1428
DISCUS_100	2569	2373	1827	1636	1503
DISCUS_150	2346	2250	1776	1694	1476
DIFFERENTPOWERS_25	3815	6022	1623	2123	2354
DIFFERENTPOWERS_50	12072	12440	6920	6567	7469
DIFFERENTPOWERS_100	23887	21327	11816	12344	15667
DIFFERENTPOWERS_150	27185	23196	12003	12858	16164
ELLIPSOIDAL_25	1918	3079	1216	1216	1354
ELLIPSOIDAL_50	7804	6986	2176	2173	2736
ELLIPSOIDAL_100	19182	14093	3833	3843	5923
ELLIPSOIDAL_150	23940	17710	4341	4518	6607
GALLAGHER21_25	1092	1649	1397	1376	1334
GALLAGHER21_50	3028	3257	3095	2740(0.93)	2080(0.83)
GALLAGHER21_100	4346	4051(0.93)	3493(0.83)	3157(0.80)	2588(0.67)
GALLAGHER21_150	2812(0.80)	2300(0.70)	2472(0.73)	2087(0.63)	1946(0.63)
GALLAGHER101_25	1142(0.93)	1768(0.93)	1209(0.93)	1252(0.93)	1232(0.93)
GALLAGHER101_50	3370(0.83)	3524(0.80)	2447(0.73)	2612(0.77)	2717(0.87)
GALLAGHER101_100	4817	4769	4235(0.93)	3651(0.87)	4028(0.90)
GALLAGHER101_150	4304	4195	3742(0.97)	3688(0.97)	3832(0.93)
GRIEWANK _25	2456	3684	1399	1345	1669
GRIEWANK _50	4822	4831	2675	2612	2676
GRIEWANK _100	7646	6444	4111	4239	4096
GRIEWANK _150	7198	6136	3897	3704	3838
GRIEWANK_ROSENBROCK_25	3070	4220	1387	1434	1947
GRIEWANK_ROSENBROCK_50	6306	6245	2696	2446	4150
GRIEWANK_ROSENBROCK_100	9049	7884	3686	3641	6283
GRIEWANK_ROSENBROCK_150	8528	7502	3463	3394	6108
ROSENBROCK_25	3494	4831	1364	1311	1915
ROSENBROCK_50	9279	9174	2842	2944	4367
ROSENBROCK_100	17258	15410	2842	5602	9713
ROSENBROCK_150	19511	18071	5628	6356	10792
RARSTIGIN_25	2486	3451(0.97)	1535	1542	1907
RARSTIGIN_50	6278	6155(0.97)	3888	3461(0.93)	3876(0.90)
RARSTIGIN_100	9052	8404	5608	5318	6810(0.97)
RARSTIGIN_150	8088	7444	4942	4861(0.97)	5742(0.93)
STEP ELLIPSOIDAL_25	666	1032	1154	1160	1092
STEP ELLIPSOIDAL_50	1273	1476	1507	1500	1419
STEP ELLIPSOIDAL_100	1676	1697	1737	1745	1696
STEP ELLIPSOIDAL_150	1529	1633	1698	1691	1600
SHARP RIDGE_25	3047	4652	1372	1329	1668
SHARP RIDGE_50	6643	6420	2322	2184	2969
SHARP RIDGE_100	9566	8344	3651	3345	4752
SHARP RIDGE_150	8976	7779	3034	2880	3891
ZAKHAROV_25	1768	2040	1355	1351	1617
ZAKHAROV_50	4141	4001	1819	1830	3488
ZAKHAROV_100	5050	6268	1344	1318	5381
ZAKHAROV_150	7134	7239	1539	1513	5002
	384022(0.99)	363819(0.99)	173392(0.99)	174401(0.98)	228652(0.97)

The diagram of Figure 3 shows the effect of the number of clusters on the performance of the algorithm. Specifically, the cases of 1, 3, 10, 20 and 30 clusters are compared. It is observed that the use of more subpopulations reduces the number of function calls, with the cases of 10 and 20 clusters showing the smallest values. On the contrary, when only 1 cluster

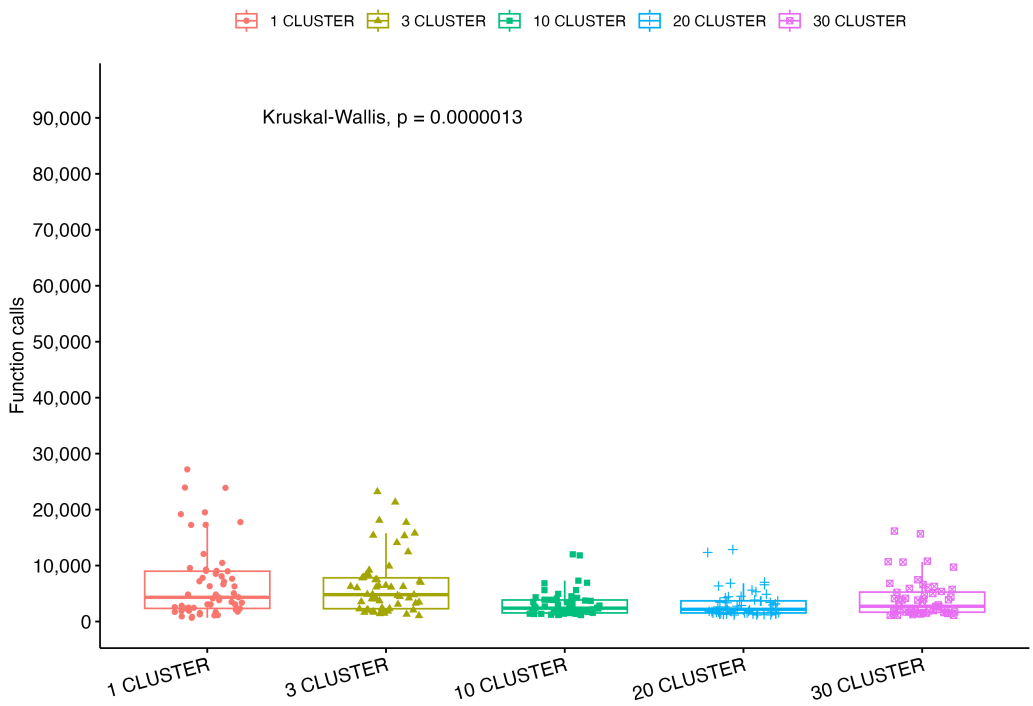

Figure 3. Impact of the Number of Clusters.

is used, the method shows greater dispersion and higher values, which indicates lower efficiency. The Kruskal–Wallis statistical test (p = 0.0000013) shows that there are statistically significant differences between different numbers of clusters. Therefore, the results show that using an appropriate number of subpopulations can improve the performance and convergence speed of the algorithm.

4.4. Impact of Propagation Mechanism

Table 4 presents the comparative evaluation of the proposed method under two different configurations, namely for PROP = 0 and PROP = 1, in terms of the number of evaluations of the objective function in a set of benchmark functions and for dimensions 25, 50, 100, 150. The PROP parameter indicates the presence or absence of the information propagation mechanism between the subpopulations: the case PROP = 0 corresponds to execution without propagation, while the case PROP = 1 corresponds to execution with propagation enabled. The results show that the activation of the propagation mechanism leads, in almost all the examined functions and dimensions, to a reduction in the number of evaluations compared to the corresponding execution without propagation. For example, in the Büche Rastrigin_150 function the number of evaluations is reduced from 22453 to 6828, in the Sharp Ridge_150 from 10708 to 3034, while in the Zakharov_150 from 9869 to 1539. The improvement is correspondingly significant in lower dimensions, which suggests that the propagation mechanism not only favors large-scale problems, but also improves the overall behavior of the method. The total number of function calls is reduced from about 493,939 in the case without propagation (PROP = 0) to about 173,392 in the case with propagation (PROP = 1). This difference corresponds to a significant reduction in the overall computational cost and confirms that information propagation between subpopulations is an important mechanism for improving the efficiency of the proposed method. Overall, the results show that the integration of the propagation mechanism

enhances the computational behavior of the algorithm, reducing the number of required objective function evaluations and improving efficiency.

Table 4. Experimental results using different propagation mechanism.

FUNCTION	PROR=0	PROR=1
ATTRACTIVE SECTOR_25	1745	1171
ATTRACTIVE SECTOR_50	2162	1549
ATTRACTIVE SECTOR_100	2414	1818
ATTRACTIVE SECTOR_150	2440	1754
BUCHE RASTRIGIN_25	8599	1606
BUCHE RASTRIGIN_50	14282	4514
BUCHE RASTRIGIN_100	20788	7302
BUCHE RASTRIGIN_150	22453(0.97)	6828
DISCUS_25	1809	1221
DISCUS_50	2022	1513
DISCUS_100	2392	1827
DISCUS_150	2349	1776
DIFFERENTPOWERS_25	9905	1623
DIFFERENTPOWERS_50	16094	6920
DIFFERENTPOWERS_100	26466	11816
DIFFERENTPOWERS_150	33123	12003
ELLIPSOIDAL_25	5003	1216
ELLIPSOIDAL_50	9973	2176
ELLIPSOIDAL_100	20082	3833
ELLIPSOIDAL_150	27981	4341
GALLAGHER21_25	2765	1397
GALLAGHER21_50	3679(0.97)	3095
GALLAGHER21_100	2044(0.53)	3493(0.83)
GALLAGHER21_150	667(0.40)	2472(0.73)
GALLAGHER101_25	2288(0.93)	1209(0.93)
GALLAGHER101_50	3504(0.83)	2447(0.73)
GALLAGHER101_100	4991(0.90)	4235(0.93)
GALLAGHER101_150	5032	3742(0.97)
GRIEWANK _25	5876	1399
GRIEWANK _50	6446	2675
GRIEWANK _100	7921	4111
GRIEWANK _150	8657	3897
GRIEWANK_ROSENBROCK_25	6456	1387
GRIEWANK_ROSENBROCK_50	8186	2696
GRIEWANK_ROSENBROCK_100	9765	3686
GRIEWANK_ROSENBROCK_150	10496	3463
ROSENBROCK_25	8468	1364
ROSENBROCK_50	12829	2842
ROSENBROCK_100	20141	2842
ROSENBROCK_150	25818	5628
RARSTIGIN_25	6901	1535
RARSTIGIN_50	8901	3888
RARSTIGIN_100	10440	5608
RARSTIGIN_150	10149	4942
STEP ELLIPSOIDAL_25	1351	1154
STEP ELLIPSOIDAL_50	1623	1507
STEP ELLIPSOIDAL_100	1769	1737
STEP ELLIPSOIDAL_150	1720	1698
SHARP RIDGE_25	7434	1372
SHARP RIDGE_50	9092	2322
SHARP RIDGE_100	10380	3651
SHARP RIDGE_150	10708	3034
ZAKHAROV_25	2631	1355
ZAKHAROV_50	4674	1819
ZAKHAROV_100	8186	1344
ZAKHAROV_150	9869	1539
	493939(0.97)	173392(0.99)

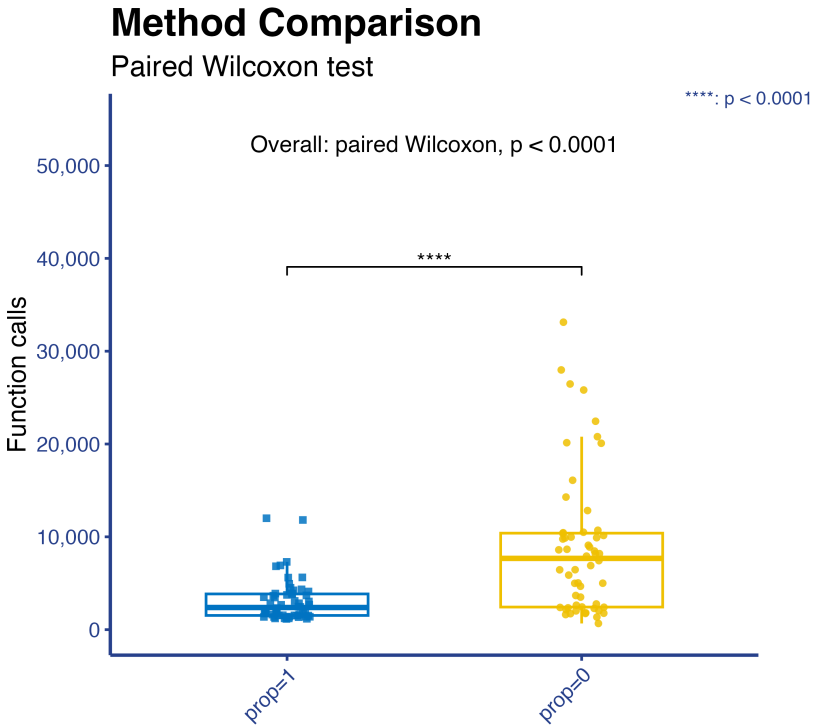

Figure 4. Impact of Propagation Mechanism.

The diagram of Figure 4 shows the comparison of the method when the propagation function is enabled and disabled. Specifically, PROP = 1 indicates that propagation is enabled, while PROP = 0 indicates that the method is executed without propagation. It is observed that when propagation is enabled (PROP = 1), the method requires significantly fewer function calls and exhibits lower dispersion, which indicates faster convergence. The comparison is performed using the paired Wilcoxon statistical test, following the conventional star notation (ns: $p > 0.05$, *: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$), where the overall result ($p < 0.0001$) shows that the difference between the two cases is significant. Therefore, the results show that enabling propagation improves the efficiency of the proposed method.

4.5. Communication Strategy with 10 Subpopulations

Table 5presents the comparative evaluation of four information dissemination strategies 1to1, 1toN, Nto1 and NtoN for 10 subpopulations, in terms of the number of evaluations of the objective function in a set of functions and for dimensions 25, 50, 100 and 150. The results show that the NtoN strategy exhibits the best computational behavior overall and leads to the smallest function calls values in the vast majority of the functions examined. For example, in the Büche Rastrigin_150 function the NtoN strategy requires 6828 function calls, compared to 18787 for the 1to1 strategy, 10340 for 1toN and 11452 for Nto1. Correspondingly, in Zakharov_150 the NtoN strategy requires only 1539 function calls, while the 1to1, 1toN and Nto1 strategies require 9158, 4352 and 4690, respectively. Therefore, the fully bidirectional information dissemination between subpopulations enhances cooperation and facilitates faster diffusion of useful information in the population. The 1toN and Nto1 strategies also show improved behavior compared to the basic 1to1 approach. The total number of function calls is 432,256 for the 1to1 strategy, 254,443 for 1toN, 274,853 for Nto1 and only 173,392 for NtoN. These results show that, for the case of 10 subpopulations, the

NtoN strategy is the most efficient choice, as it achieves the lowest overall computational requirement.

https://doi.org/10.3390/1010000

Table 5. Experimental results using different communication strategy and 10 subpopulations.

FUNCTION	1to1	1toN	Nto1	NtoN
ATTRACTIVE SECTOR_25	1602	1348	1349	1171
ATTRACTIVE SECTOR_50	2027	1648	1698	1549
ATTRACTIVE SECTOR_100	2271	1912	1944	1818
ATTRACTIVE SECTOR_150	2282	1859	1914	1754
BUCHE RASTRIGIN_25	7204	3327	3521	1606
BUCHE RASTRIGIN_50	11493(0.93)	6203	6789(0.97)	4514
BUCHE RASTRIGIN_100	17882	10472	11567	7302
BUCHE RASTRIGIN_150	18787(0.93)	10340	11452(0.97)	6828
DISCUS_25	1726	1359	1392	1221
DISCUS_50	1923	1517	1706	1513
DISCUS_100	2059	1911	1691	1827
DISCUS_150	2088	1822	1743	1776
DIFFERENTPOWERS_25	8481	3981	4630	1623
DIFFERENTPOWERS_50	14513	9641	9649	6920
DIFFERENTPOWERS_100	23558	15731	17144	11816
DIFFERENTPOWERS_150	28561	16633	18553	12003
ELLIPSOIDAL_25	4112	2151	2353	1216
ELLIPSOIDAL_50	8553	3614	4283	2176
ELLIPSOIDAL_100	16965	6082	7615	3833
ELLIPSOIDAL_150	22798	7751	9556	4341
GALLAGHER21_25	2319	1692	1689	1397
GALLAGHER21_50	3201(0.93)	3137(0.97)	2914(0.93)	3095
GALLAGHER21_100	2419(0.60)	3273(0.77)	2476(0.63)	3493(0.83)
GALLAGHER21_150	1079(0.47)	1507(0.53)	1245(0.50)	2472(0.73)
GALLAGHER101_25	2148	1580(0.97)	1636	1209(0.93)
GALLAGHER101_50	3777(0.97)	3572(0.97)	3335(0.93)	2447(0.73)
GALLAGHER101_100	5272	4891	4729(0.97)	4235(0.93)
GALLAGHER101_150	4785	4282	4429	3742(0.97)
GRIEWANK _25	5277	2780	3232	1399
GRIEWANK _50	5682	3384	3679	2675
GRIEWANK _100	6813	4314	4799	4111
GRIEWANK _150	7549	4274	4541(0.87)	3897
GRIEWANK_ROSENBROCK_25	5722	2966	3467(0.97)	1387
GRIEWANK_ROSENBROCK_50	7311	4285	4700(0.97)	2696
GRIEWANK_ROSENBROCK_100	8723	5430	6023	3686
GRIEWANK_ROSENBROCK_150	9517	5367	5802	3463
ROSENBROCK_25	7668	3471	4288	1364
ROSENBROCK_50	11170	5907	6752	2842
ROSENBROCK_100	17250	10773	11768	2842
ROSENBROCK_150	23256	12622	14393	5628
RARSTIGIN_25	5500	2935	3295	1535
RARSTIGIN_50	7523	5364	4541	3888
RARSTIGIN_100	8597(0.97)	6728	6790	5608
RARSTIGIN_150	8784(0.97)	6025(0.97)	6138	4942
STEP ELLIPSOIDAL_25	1328	1205	1207	1154
STEP ELLIPSOIDAL_50	1593	1530	1524	1507
STEP ELLIPSOIDAL_100	1742	1735	1743	1737
STEP ELLIPSOIDAL_150	1713	1701	1723	1698
SHARP RIDGE_25	6833	3248	3390	1372
SHARP RIDGE_50	7629	3917	4348	2322
SHARP RIDGE_100	8889	4814	5363	3651
SHARP RIDGE_150	9656	4522	5010	3034
ZAKHAROV_25	2407	1682	1850	1355
ZAKHAROV_50	4004	2579	2651	1819
ZAKHAROV_100	7077	3297	4144	1344
ZAKHAROV_150	9158	4352	4690	1539
	432256(0.98)	254443(0.99)	274853(0.98)	173392(0.99)

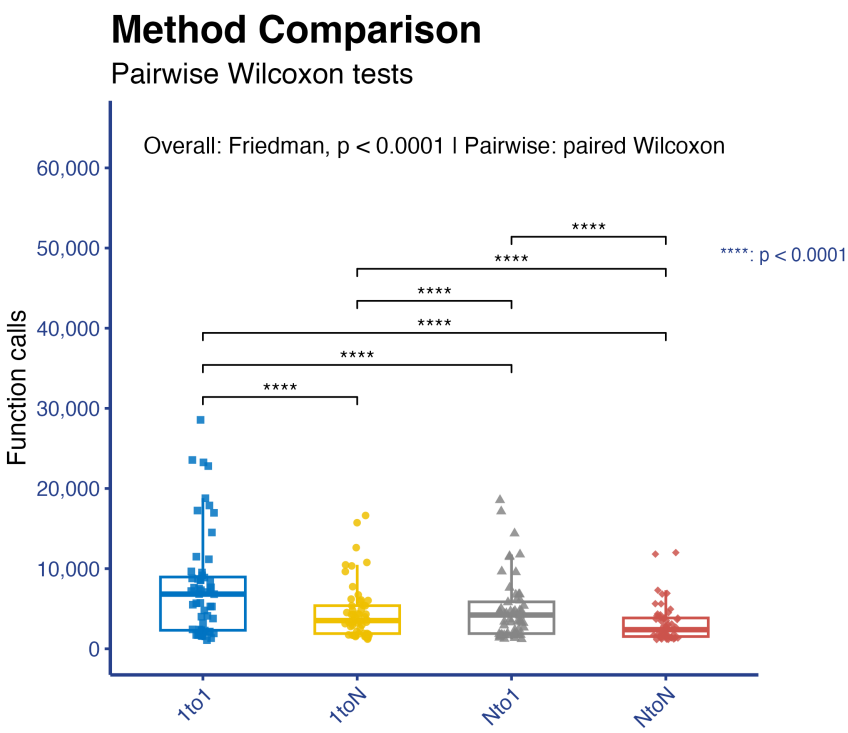

Figure 5. Communication Strategy with 10 Subpopulations.

The diagram of figure 5 shows the comparison of different communication modes between subpopulations when 10 clusters are used, based on the number of function calls. The configurations examined are 1to1, 1toN, Nto1 and NtoN, which express different information dissemination schemes between subpopulations. It is observed that the NtoN configuration presents the lowest values and the smallest dispersion, which indicates faster convergence of the algorithm. On the contrary, the 1to1 case presents a higher number of function calls. The overall statistical analysis was performed with the Friedman test ($p < 0.0001$), while the individual comparisons were performed with paired Wilcoxon tests, following the conventional star notation (ns: $p > 0.05$, *: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$). The presence of **** ($p < 0.0001$) indicates that the differences between the methods are statistically significant. The results suggest that the exchange of information between all subpopulations is significant.

4.6. Communication Strategy with 20 Subpopulations

Table 6 presents the comparative evaluation of four information dissemination strategies 1to1, 1toN, Nto1 and NtoN for 20 subpopulations, in terms of the number of evaluations of the objective function in a set of functions and for dimensions 25, 50, 100 and 150. The results show that, also in the case of 20 subpopulations, the NtoN strategy displays the overall most efficient behavior, as it achieves the lowest or among the lowest function calls values. Indicatively, in the Rosenbrock_150 function the NtoN strategy requires 6356 function calls, compared to 23645 for 1to1, 8431 for 1toN and 11263 for Nto1. Accordingly, in Rastrigin_150 the required evaluations are reduced to 4861 for NtoN, compared to 8674, 5954 and 5671 for the 1to1, 1toN and Nto1 strategies, respectively. These results suggest that the fully cooperative form of propagation significantly accelerates the diffusion of useful information between subpopulations and facilitates convergence towards better regions of the search space. The 1toN and Nto1 strategies also show improved behavior compared

to the basic 1to1 approach, which confirms that even partial information propagation is beneficial for the search process. The total number of function calls is 414,978 for the 1to1 strategy, 195,980 for 1toN, 222,989 for Nto1 and only 174,401 for NtoN. These results show that, for the case of 20 subpopulations, the NtoN strategy is again the most efficient choice, as it achieves the lowest overall computational requirement.

Table 6. Experimental results using different communication strategy and 20 subpopulations.

FUNCTION	1to1	1toN	Nto1	NtoN
ATTRACTIVE SECTOR_25	1663	1317	1271	1170
ATTRACTIVE SECTOR_50	2026	1582	1644	1534
ATTRACTIVE SECTOR_100	2247	1826	1893	1809
ATTRACTIVE SECTOR_150	2231	1797	1898	1754
BUCHE RASTRIGIN_25	7185	2674	2768	1578
BUCHE RASTRIGIN_50	11071(0.97)	5436(0.97)	5108(0.87)	4420(0.97)
BUCHE RASTRIGIN_100	15630(0.90)	7698(0.97)	8794(0.83)	7006(0.97)
BUCHE RASTRIGIN_150	17743(0.97)	7296(0.90)	8690(0.87)	6837(0.97)
DISCUS_25	1721	1222	1330	1153
DISCUS_50	1833	1413	1368	1549
DISCUS_100	1803	1642	1555	1636
DISCUS_150	1927	1601	1572	1694
DIFFERENTPOWERS_25	9370	3382	3277	2123
DIFFERENTPOWERS_50	14912	7296	7985	6567
DIFFERENTPOWERS_100	23185	12713	14072	12344
DIFFERENTPOWERS_150	27829	13781	15957	12858
ELLIPSOIDAL_25	4242	1600	1805	1216
ELLIPSOIDAL_50	7652	2879	3370	2173
ELLIPSOIDAL_100	14835	4172	5690	3843
ELLIPSOIDAL_150	20040	5562	6949	4518
GALLAGHER21_25	2347	1542	1582	1376
GALLAGHER21_50	2609(0.80)	2656(0.93)	2031(0.80)	2740(0.93)
GALLAGHER21_100	1250(0.40)	2255(0.60)	1725(0.50)	3157(0.80)
GALLAGHER21_150	654(0.40)	1343(0.50)	1019(0.47)	2087(0.63)
GALLAGHER101_25	2180	1383(0.97)	1630	1252(0.93)
GALLAGHER101_50	3554(0.90)	2894(0.87)	2899(0.90)	2612(0.77)
GALLAGHER101_100	4806(0.87)	3942(0.83)	3927(0.83)	3651(0.87)
GALLAGHER101_150	5034	4093(0.97)	3804(0.93)	3688(0.97)
GRIEWANK _25	5255	2219	2341	1345
GRIEWANK _50	5545	3050	3292	2612
GRIEWANK _100	6280	3760	3823	4239
GRIEWANK _150	6933	3596	3887	3704
GRIEWANK_ROSENBROCK_25	5566	1851	2653	1434
GRIEWANK_ROSENBROCK_50	7156	3145	3754	2446
GRIEWANK_ROSENBROCK_100	8461	3983	4858	3641
GRIEWANK_ROSENBROCK_150	8923	3773	4887	3394
ROSENBROCK_25	7197	1876	2995	1311
ROSENBROCK_50	10930	3825	5192	2944
ROSENBROCK_100	17190	6262	9295	5602
ROSENBROCK_150	23645	8431	11263	6356
RARSTIGIN_25	5727	1985	2346	1542
RARSTIGIN_50	6901(0.93)	4141(0.97)	4157(0.93)	3461(0.93)
RARSTIGIN_100	8313(0.93)	5491(0.90)	5433(0.87)	5318
RARSTIGIN_150	8674(0.97)	5954(0.90)	5671(0.97)	4861(0.97)
STEP ELLIPSOIDAL_25	1342	1238	1198	1160
STEP ELLIPSOIDAL_50	1557	1516	1539	1500
STEP ELLIPSOIDAL_100	1772	1733	1733	1745
STEP ELLIPSOIDAL_150	1727	1705	1706	1691
SHARP RIDGE_25	6774	2150	2645	1329
SHARP RIDGE_50	7431	2899	3598	2184
SHARP RIDGE_100	8358	4010	4411	3345
SHARP RIDGE_150	8713	3680	4186	2880
ZAKHAROV_25	2338	1391	1557	1351
ZAKHAROV_50	4005	1845	2371	1830
ZAKHAROV_100	6815	1687	2836	1318
ZAKHAROV_150	9871	1787	3749	1513
	414978(0.97)	195980(0.97)	222989(0.96)	174401(0.98)

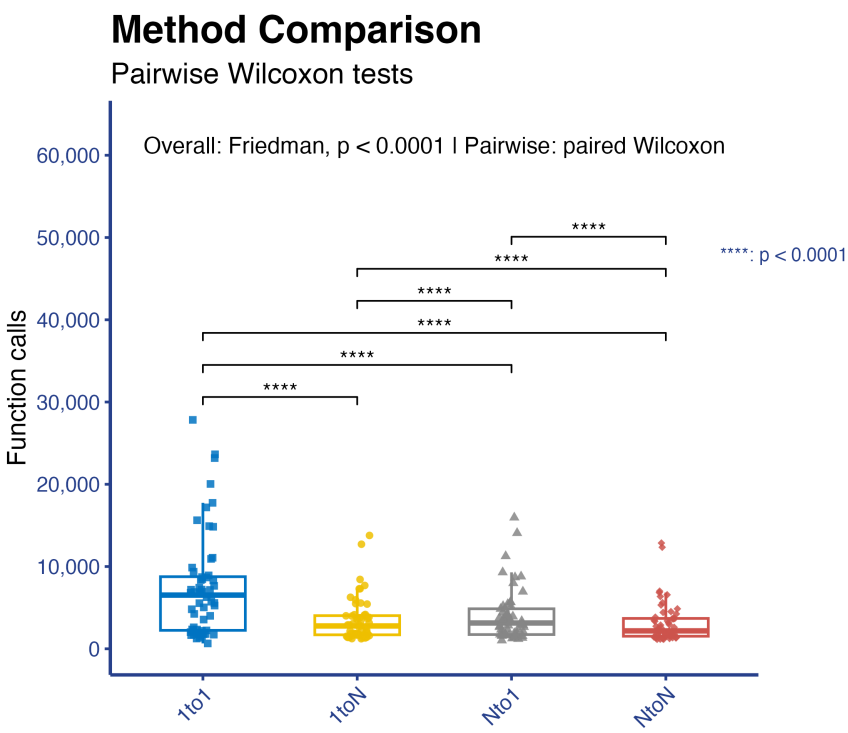

Figure 6. Communication Strategy with 20 Subpopulations.

The diagram of figure 6 shows the comparison of different communication modes between subpopulations when 20 clusters are used, based on the number of function calls. The configurations examined are 1to1, 1toN, Nto1 and NtoN, which express different information dissemination schemes between subpopulations. It is observed that the NtoN configuration presents the lowest values and the smallest dispersion, which indicates faster convergence of the algorithm. On the contrary, the 1to1 case presents a higher number of function calls. The overall statistical analysis was performed with the Friedman test ($p < 0.0001$), while the individual comparisons were performed with paired Wilcoxon tests, following the conventional star notation (ns: $p > 0.05$, *: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$). The presence of **** ($p < 0.0001$) indicates that the differences between the methods are statistically significant. The results suggest that the exchange of information between all subpopulations is significant.

4.7 The proposed method in comparison with other methods

Table 7 presents the comparative evaluation of the proposed method against established metaheuristic algorithms, namely GA, DE, GWO, ParallelDE, SAOP, BGWO and WOA, in terms of the number of evaluations of the objective function in a set of benchmark functions and for dimensions 25, 50, 100 and 150. The results show that the proposed method shows the best performance most of the time. Indicatively, in the Zakharov_150 function the proposed method requires 1539 evaluations, compared to 38858 for GA, 14744 for DE, 3251 for GWO, 5613 for ParallelDE, 4341 for SAOP, 3333 for BGWO and 11943 for WOA. Correspondingly, on Sharp Ridge_150 the proposed method achieves 3034 function calls, compared to 12266, 12231, 25399, 5614, 4055, 24923 and 8022, respectively. Particularly important is the fact that the proposed approach maintains its efficiency not only in complex and multimodal problems, but also in functions of different structure, presenting consistently low computational cost across the entire range of dimensions. The total number

of function calls of the proposed method amounts to 173,392, compared to 777,899 for GA, 762,504 for DE, 772,866 for GWO, 314,144 for ParallelDE, 263,370 for SAOP, 724,617 for BGWO and 371,962 for WOA. In conclusion, it follows that the combination of the cooperative subpopulation structure, the information dissemination mechanism and the overall parallelized search strategy substantially enhances the efficiency of the algorithm, making it particularly suitable for optimization and high-dimensional problems.

Table 7. Comparing the proposed method against other methods.

FUNCTION	GA	DE	GWO	PARALLELDE	SAOP	BGWO	WOA	PROPOSED
ATTRACTIVE SECTOR_25	2281	2234	3772	5594	1715	3239	1944	1171
ATTRACTIVE SECTOR_50	2297	2298	12556	5611	2106	17575	4646	1549
ATTRACTIVE SECTOR_100	2344	2273	29011	5623	2691	28735	6739	1818
ATTRACTIVE SECTOR_150	2345	2255	25348	5614	2618	24872	6070	1754
BUCHE RASTRIGIN_25	11233(0.93)	11417(0.93)	4214	5592	2183(0.93)	1989(0.93)	3009(0.97)	1606
BUCHE RASTRIGIN_50	17589(0.57)	21873(0.57)	13509(0.90)	5610	4657(0.57)	4033(0.57)	14163(0.70)	4514
BUCHE RASTRIGIN_100	32881(0.30)	39401(0.30)	23867(0.80)	5622	7806(0.30)	7564(0.30)	7293(0.97)	7302
BUCHE RASTRIGIN_150	36444(0.40)	48017(0.40)	22139(0.87)	5611	7731(0.40)	6472(0.40)	5699	6828
DISCUS_25	2715	2612	4979	5596	1747	4266	2505	1221
DISCUS_50	2714	2701	19111	5614	2472	18438	8241	1513
DISCUS_100	2752	2693	29094	5624	2722	28740	12795	1827
DISCUS_150	2750	2671	25351	5614	2483	24876	11185	1776
DIFFERENTPOWERS_25	13758	14154	3580	5595	2456	2728	1864	1623
DIFFERENTPOWERS_50	19777	22093	10497	5612	8075	12334	4211	6920
DIFFERENTPOWERS_100	28988	29193	25884	5624	13699	27204	6439	11816
DIFFERENTPOWERS_150	34372	34490	25642	5613	15676	25413	5930	12003
ELLIPSOIDAL_25	5930	5778	3598	5593	1847	2778	1843	1216
ELLIPSOIDAL_50	10583	11023	10624	5611	2979	13691	4127	2176
ELLIPSOIDAL_100	20604	20345	25068	5623	5137	28976	6014	3833
ELLIPSOIDAL_150	28556	27643	25642	5612	5893	25171	5653	4341
GALLAGHER21_25	3118(0.93)	2867(0.93)	3214(0.93)	5595	2206(0.93)	2115(0.93)	3978(0.93)	1397
GALLAGHER21_50	3272(0.57)	3566(0.57)	6234(0.57)	5612	8557(0.57)	6590(0.57)	14831(0.57)	3095
GALLAGHER21_100	1693(0.30)	1621(0.30)	2901(0.30)	5622	1691(0.30)	8781(0.30)	1502(0.30)	3493(0.83)
GALLAGHER21_150	1682(0.40)	1617(0.40)	2898(0.40)	5613	1671(0.40)	1703(0.40)	1500(0.40)	2472(0.73)
GALLAGHER101_25	3083(0.93)	3030(0.93)	3106(0.93)	5593	2068	1913(0.93)	3531(0.93)	1209(0.93)
GALLAGHER101_50	6150(0.57)	4617(0.57)	5873(0.57)	5602	6932	7429(0.57)	13266(0.57)	2447(0.73)
GALLAGHER101_100	6656(0.30)	4340(0.30)	8356(0.30)	5606	18872	12116(0.30)	18537(0.30)	4235(0.93)
GALLAGHER101_150	6122(0.40)	3236(0.40)	7496(0.40)	5601	19066	14717(0.40)	15220(0.40)	3742(0.97)
GRIEWANK _25	9514	9655	3695(0.97)	5594	2237	2878(0.97)	1893	1399
GRIEWANK _50	5511	5147	10342(0.97)	5610	3132	12370(0.83)	4184	2675
GRIEWANK _100	5078	4141	22295	5623	4053	27343(0.87)	6532(0.97)	4111
GRIEWANK _150	5284	4131	25374	5612	3934	24895(0.93)	6098(0.97)	3897
GRIEWANK_ROSENBROCK_25	16098	17837	3429	5595	2259	2435	1740	1387
GRIEWANK_ROSENBROCK_50	22598	27316	8659	5612	4413	9859	3361	2696
GRIEWANK_ROSENBROCK_100	30869	35225	19410	5624	5856	21301	4784	3686
GRIEWANK_ROSENBROCK_150	37686	40126	22950	5613	6664	23047	4450	3463
ROSENBROCK_25	15148	15199	3760	5595	2217	3320	1925	1364
ROSENBROCK_50	23663	25189	12789	5612	4238	18057	4611	2842
ROSENBROCK_100	40370	39254	29103	5623	7197	28756	6786	2842
ROSENBROCK_150	53927	51902	25367	5613	8677	24893	6076	5628
RARSTIGIN_25	9348(0.93)	9799(0.93)	2959(0.93)	5594	2067(0.93)	2007(0.93)	3007(0.97)	1535
RARSTIGIN_50	11414(0.57)	13738(0.57)	13386(0.87)	5610	3871(0.57)	4022(0.57)	15194(0.67)	3888
RARSTIGIN_100	12898(0.30)	17722(0.30)	22715(0.77)	5622	4840(0.30)	7611(0.30)	10638(0.87)	5608
RARSTIGIN_150	14392(0.40)	19506(0.40)	18708(0.73)	5612	4693(0.40)	6693(0.40)	6827(0.97)	4942
STEP ELLIPSOIDAL_25	1718(0.93)	1664(0.93)	3078	5593	1806(0.93)	1831	1590	1154
STEP ELLIPSOIDAL_50	2201(0.57)	1661(0.57)	5163	5610	2369(0.57)	3615(0.90)	2273(0.93)	1507
STEP ELLIPSOIDAL_100	2028(0.30)	1629(0.30)	10101(0.97)	5622	2355(0.30)	6624(0.57)	2708(0.97)	1737
STEP ELLIPSOIDAL_150	1752(0.40)	1617(0.40)	12778(0.97)	5612	1977(0.40)	6730(0.44)	2527(0.97)	1698
SHARP RIDGE_25	11324	11800	4098	5596	2124	4087	2110	1372
SHARP RIDGE_50	11454	12866	16581	5612	3106	18454	5907	2322
SHARP RIDGE_100	12105	12682	29123	5624	3995	28790	8936	3651
SHARP RIDGE_150	12266	12231	25399	5614	4055	24923	8022	3034
ZAKHAROV_25	5760	4939	8426	5598	2177	9075	10581	1355
ZAKHAROV_50	15253	8154	23034	5611	3083	27447	25061	1819
ZAKHAROV_100	36693	12572	3329	5623	3878	5763	9463	1344
ZAKHAROV_150	38858	14744	3251	5613	4341	3333	11943	1539
	777899(0.84)	762504(0.84)	772866(0.91)	314144	263370(0.84)	724617(0.85)	371962(0.92)	173392(0.99)

The diagram of figure 7 shows the comparison of the performance of various optimization algorithms in terms of the number of function calls. Specifically, the comparison of the algorithms GENETIC, DE, GWO, PARALLELDE, SAOP, BGWO, PROPOSED and WOA. It is observed that the proposed algorithm presents the smallest values of function calls and less variance compared to most algorithms, therefore it has faster convergence.The GENETIC, DE and BGWO algorithms show greater dispersion and higher values, while PARALLELDE shows low values. The Kruskal–Wallis statistical test ($p < 2.2e{-}16$) shows that there are statistically significant differences between the algorithms. Overall, the results indicate that the proposed method is superior in terms of efficiency.

4.8 Practical problems

To further examine the practical efficiency and scalability of the proposed optimization algorithm, two real-world engineering design problems were investigated: the GasCycle

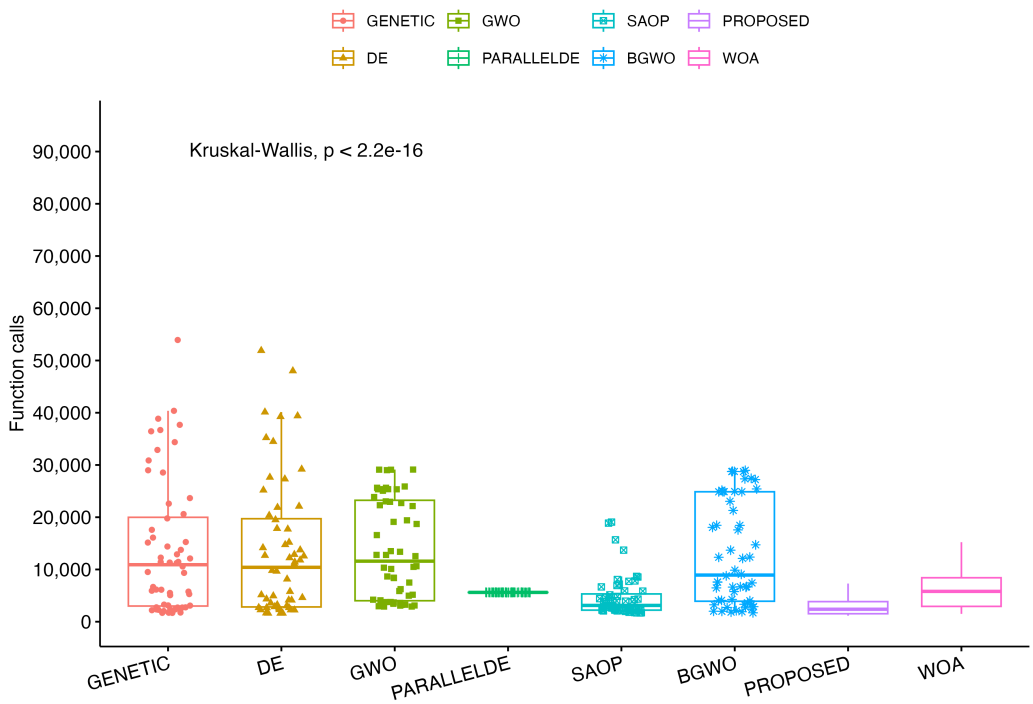

Figure 7. The proposed method in comparison with others.

[49] and the Tandem Queueing System [50]. These problems were selected because they differ significantly in mathematical formulation and computational complexity, providing a comprehensive framework for evaluating the algorithm's performance under diverse and realistic conditions.

Each problem was tested across multiple dimensional configurations, ranging from 25 to 500 variables, in order to assess how the algorithm behaves as the search space becomes more complex. For every configuration, the execution time in seconds was recorded as the main performance indicator. This experimental setup enables a direct comparison of how computational efficiency changes with increasing dimensionality.

- **GasCycle Thermal Cycle**
  Vars: $\boldsymbol{x} = [T_1, T_3, P_1, P_3]^\top$. $\quad r = P_3/P_1,\ \gamma = 1.4$.

$$\eta(\boldsymbol{x}) = 1 - r^{-(\gamma-1)/\gamma}\,\frac{T_1}{T_3}, \qquad \min_{\boldsymbol{x}} f(\boldsymbol{x}) = -\eta(\boldsymbol{x}).$$

  Bounds: $300 \le T_1 \le 1500,\ 1200 \le T_3 \le 2000,\ 1 \le P_1, P_3 \le 20$.
  Penalty: infeasible $\Rightarrow f = 10^{20}$.
  The GasCycle scenario presents a more computationally demanding optimization problem, allowing a clearer assessment of algorithmic scalability under increased complexity.

The diagram of figure 8 outlines the comparison of the number of calls of the objective function with respect to the dimension of the problem for the GasCycle problem. It is observed that most methods generally show a constant or slightly decreasing trend in the number of function calls as the dimension increases. The PARALLELDE algorithm maintains a significantly higher number of calls in all dimensions. GENETIC, GWO, BGWO and DE move in medium value levels, presenting mild fluctuations. The proposed method Proposed (ParallelBGWO) displays one of the lowest values in the number of calls

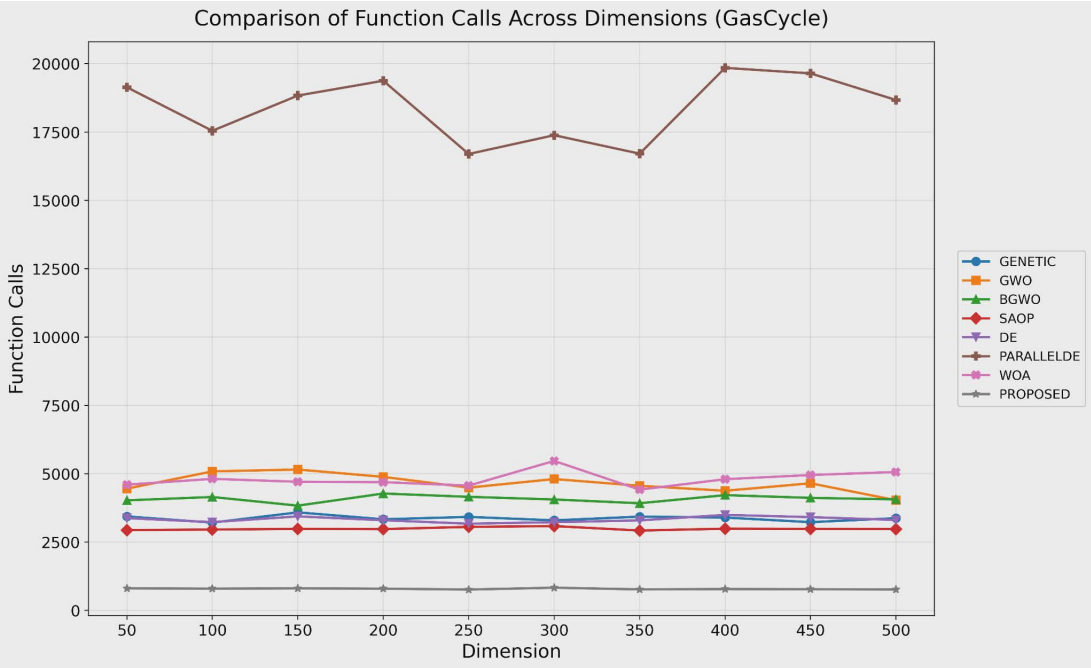

Figure 8. Comparison of Function Calls Across Dimensions (GasCycle).

and maintains a relatively constant and controllable behavior throughout the range of dimensions.

The diagram of figure 9 shows the comparison of the execution time of the algorithms with respect to the dimension of the problem for the GasCycle problem. It is observed that the execution time increases progressively with increasing dimension for all methods, which is expected due to the increased computational complexity. PARALLELDE shows significantly longer execution times compared to the other algorithms. GENETIC, GWO, BGWO, SAOP, DE and WOA show a similar scaling, with a relatively smooth and linear increase in time. The proposed method (ParallelBGWO) follows a corresponding incremental path, maintaining execution times comparable to most algorithms of the same category and clearly lower than PARALLELDE.

- **Tandem Space Trajectory** (MGA-1DSM, EVEEJ + 2×Saturn)
  Vars **(**D=18**):** $\boldsymbol{x} = [t_0, T_1, T_2, T_3, T_4, T_{5A}, T_{5B}, s_1, s_2, s_3, s_4, s_{5A}, s_{5B}, r_p, k_{A1}, k_{A2}, k_{B1}, k_{B2}]^\top$.

$$\begin{aligned} &7000 \le t_0 \le 10000, \\ &30 \le T_1 \le 500,\ 30 \le T_2 \le 600,\ 30 \le T_3 \le 1200, \\ &30 \le T_4 \le 1600,\ 30 \le T_{5A}, T_{5B} \le 2000, \\ &0 \le s_{1..4}, s_{5A}, s_{5B}, r_p, k_{A1}, k_{A2}, k_{B1}, k_{B2} \le 1. \end{aligned}$$

  Objective:

$$\min_{\boldsymbol{x}}\ \Delta V_{\text{tot}} = \Delta V_{\text{launch}}(T_1) + \Delta V_{\text{legs}}(T_1{:}T_4) + \Delta V_A + \Delta V_B + \Delta V_{\text{DSM}}(\boldsymbol{s}, r_p) - G_{\text{GA}} - G_J + P_{\text{hard}} + P_{\text{soft}},$$

$$P_{\text{soft}} = \beta \max\Big\{0,\ (T_1 + \cdots + T_4 + \tfrac{1}{2}(T_{5A} + T_{5B})) - 3500\Big\}.$$

  Notes: ΔV_{launch} decreases (log-like) in T_1 ($\ge$ 6 km/s floor), leg/branch costs decrease with TOF.

The diagram of figure 10 shows the comparison of the number of calls of the objective function with respect to the dimension of the problem for the algorithms under consideration. It is observed that the number of function calls remains relatively constant for most methods as the dimension increases, with small fluctuations per case. The BGWO algorithm displays the highest number of calls in almost all dimensions, maintaining high

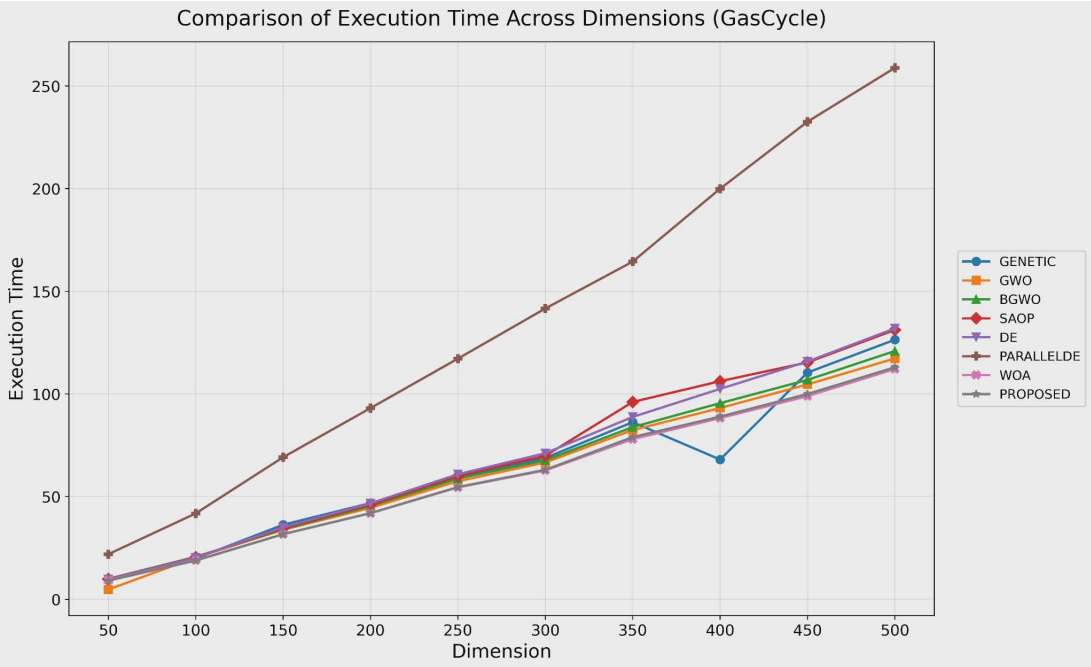

Figure 9. Comparison of Execution Time Across Dimensions (GasCycle).

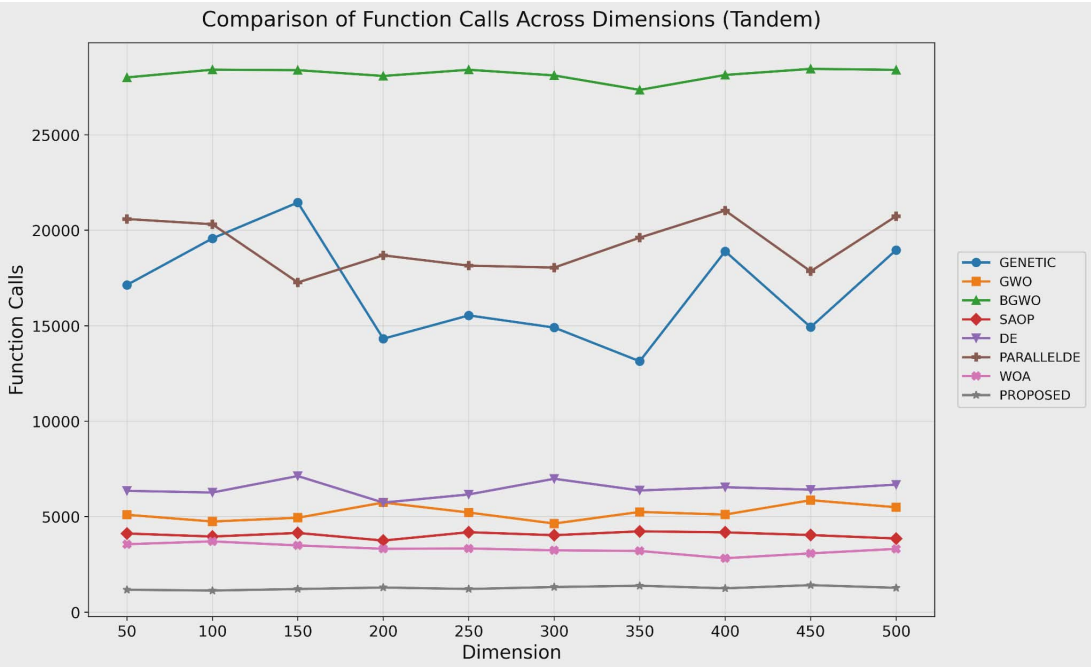

Figure 10. Comparison of Function Calls Across Dimensions (Tandem).

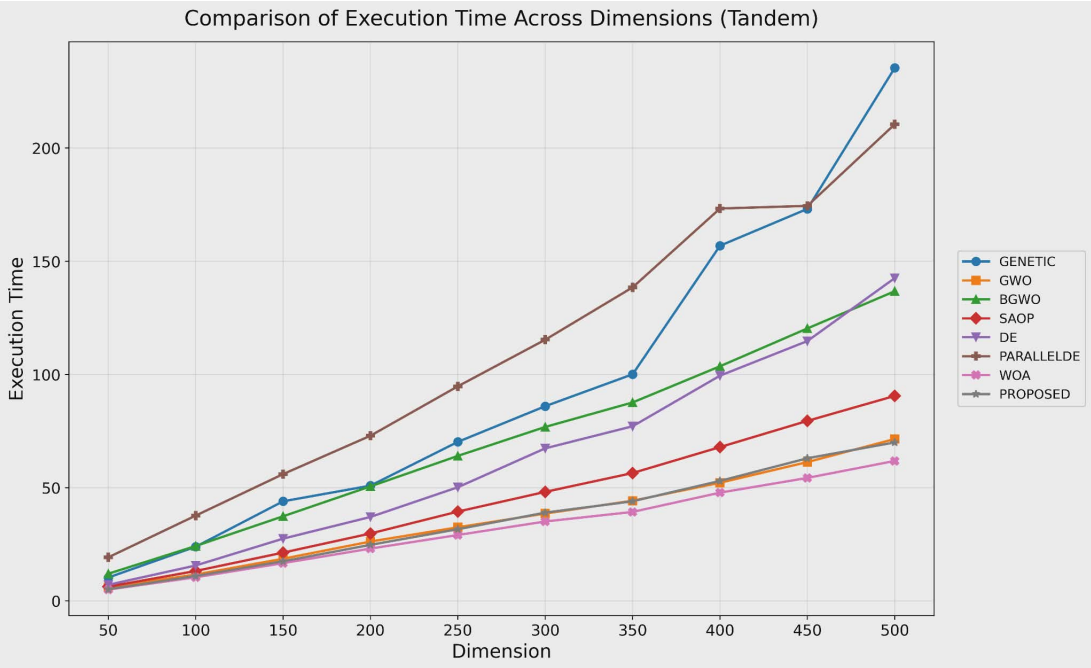

Figure 11. Comparison of Execution Time Across Dimensions (Tandem).

values with small changes. GENETIC and PARALLELDE exhibit greater fluctuations. In contrast, GWO, SAOP, DE and WOA move at lower call levels, with relatively smooth behavior. The proposed method (ParallelBGWO) exhibits among the lowest values in the number of calls and maintains a stable behavior as the dimension increases.

The diagram of figure 11 shows the comparison of the execution time of various algorithms with respect to the dimension of the problem. We observe that as the dimension increases, the execution time for all algorithms also increases. The proposed method (ParallelBGWO) clearly exhibits better scaling behavior compared to the classic BGWO and other algorithms, maintaining lower execution times in all dimensions, especially in large dimensions (400–500). In contrast, algorithms such as Genetic and ParallelDE show a sharp increase in execution time in high dimensions, which8 indicates lower computational efficiency on large problems. Overall, the results demonstrate that the proposed ParallelBGWO achieves a better balance between performance and computational cost, making it a suitable choice for large-scale problems.

4.9 Impact of the Subpopulation Termination Criterion on Practical Problems

The graph of figure 12 shows the effect of different subpopulation termination criteria (ANY, MAJORITY, ALL) on the number of objective function evaluations for the GasCycle problem, in different dimensions. From the results, it is observed that the ANY strategy presents the smallest number of function calls in all dimensions. This shows that a more flexible termination criterion can significantly reduce the computational cost. In contrast, the MAJORITY strategy requires a larger number of evaluations, as termination is triggered only when the majority of subpopulations are completed. The ALL strategy displays the highest values, since it requires the completion of all subpopulations before termination. Therefore, the results show that the more stringent the termination criterion, the higher the overall computational cost.

In the Tandem problem, a similar behavior is observed regarding the effect of the termination criteria on the number of evaluations of the objective function. The ANY strategy also presents the lowest function calls values in all dimensions, thus it is a more flexible termination criterion that can significantly reduce the computational cost. The MAJORITY

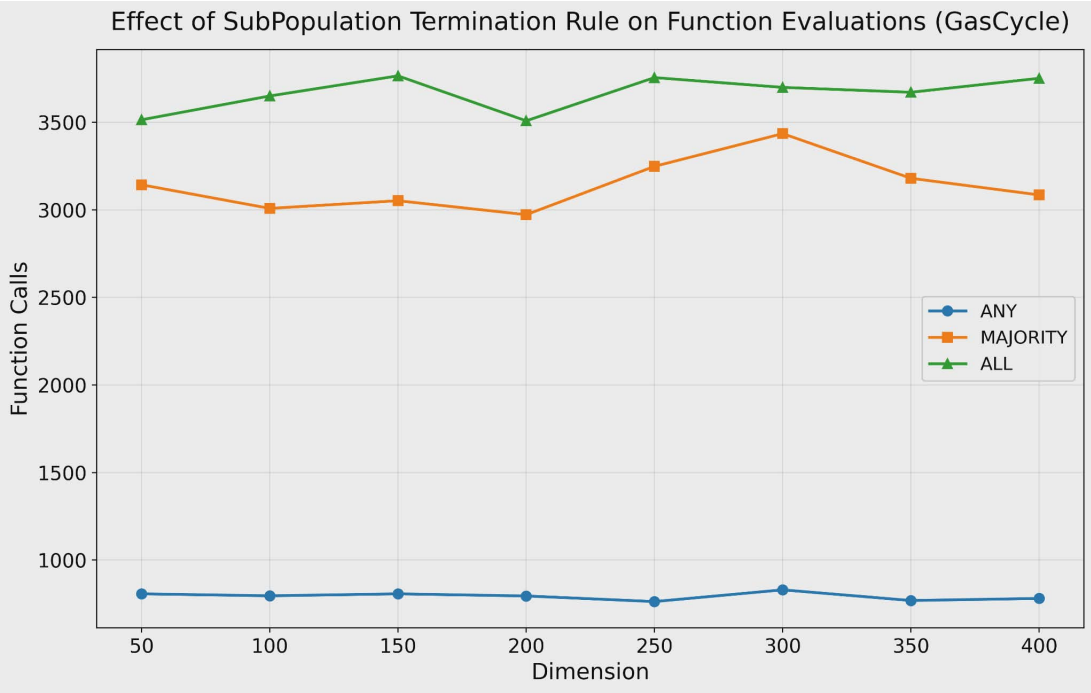

Figure 12. Effect of SubPopulation Termination Rule on Function Evaluations (GasCycle).

strategy presents greater variability and generally requires more evaluations, while the ALL strategy leads to the largest function calls values, as the search process continues until all subpopulations are completed. Therefore, the stringency of the termination criterion significantly affects the efficiency of the algorithm, with the ANY strategy offering the most computationally efficient approach. The experimental results for this case are shown graphically in Figure 13.

4.10 Neural Network Training Experiments

To validate the effectiveness of the proposed method, it was used to train artificial neural networks [53,54] for some classification problems located in various databases in the Internet, such as:

1. The UCI database, https://archive.ics.uci.edu/(accessed on 28 March 2026)[51]
2. The Keel website, https://sci2s.ugr.es/keel/datasets.php(accessed on 28 March 2026)[52].
3. The Statlib URL https://lib.stat.cmu.edu/datasets/index(accessed on 28 March 2026).

From these databases the following datasets were obtained:

1. **Circular** dataset, that contains artificially generated data.
2. **Heart** dataset [55], a medical dataset used for the prediction of heart diseases.
3. **Ionosphere** dataset, a climate dataset [56,57].
4. **Liverdisorder** dataset [58], a medical dataset.
5. **Lymography** dataset[59].
6. **Pima** dataset [60].
7. **Spiral** dataset, which is an artificial dataset.
8. **Statheart**, a dataset related to the detection of heart diseases.
9. **Wdbc** dataset [61].
10. **Wine** dataset, used for the detection of quality of wines [62,63].

All experiments were conducted 30 times, each with a different random seed. To validate the results, the widely used ten-fold cross-validation method was applied. For the classification

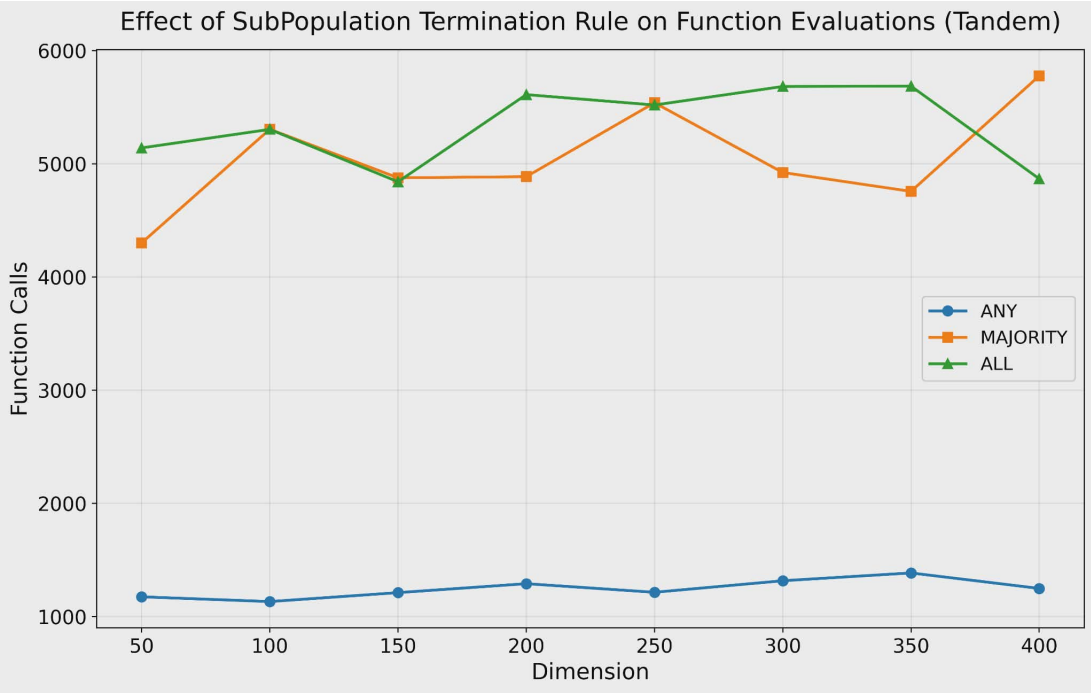

Figure 13. Effect of SubPopulation Termination Rule on Function Evaluations (Tandem).

datasets, the mean classification error is presented, computed according to the following formula:

$$E_C(N(\vec{x}, \vec{w})) = 100 \times \frac{\sum_{i=1}^{N} (\text{class}(N(\vec{x_i}, \vec{w})) - y_i)}{N} \tag{2}$$

In this equation, the set T denotes the used test set. In the experiments the following methods was used:

1. Adam, where the ADAM optimizer [64] is used to train a neural network with 10 processing nodes.
2. Bfgs, where the BFGS optimizer was incorporated to train a neural network with 10 processing nodes.
3. Genetic, where a neural network [14,15] was used to train a neural network with 10 processing nodes.
4. Proposed, where the described method is used to train a neural network with 10 processing nodes.

The results obtained by the previously mention methods are presented in Table 8.

Table 8. Average Classification Error Rates on some classification datasets using a series of methods.

DATASET	ADAM	BFGS	GENETIC	PROPOSED
Circular	19.95%	6.08%	5.99%	4.30%
Heart	38.53%	39.44%	28.34%	18.17%
Ionosphere	16.64%	15.29%	15.14%	15.00%
Liverdisorder	41.53%	42.59%	31.11%	32.89%
Lymography	39.79%	35.43%	28.42%	24.60%
Pima	34.85%	35.59%	32.19%	25.25%
Spiral	47.67%	47.99%	48.66%	41.80%
Statheart	44.04%	39.65%	27.25%	20.62%
Wine	29.40%	59.71%	19.20%	11.24%
Wdbc	35.35%	29.91%	8.56%	4.50%
AVERAGE	**34.78%**	**35.17%**	**26.26%**	**19.84%**

Table 8 indicates that PROPOSED is the most effective method overall in terms of classification error rate, since it achieves the lowest average error, 19.84%, compared with 26.26% for GENETIC, 34.78% for ADAM, and 35.17% for BFGS. Because lower values correspond to better performance, this result demonstrates a clear overall superiority of PROPOSED over both the two gradient-based baselines and the evolutionary GENETIC approach. This advantage is not marginal but substantial, suggesting that the proposed scheme attains a more favorable balance between adaptation and generalization across the examined classification datasets.

At the individual dataset level, PROPOSED ranks first in 9 out of the 10 datasets, yielding the lowest error on Circular, Heart, Ionosphere, Lymography, Pima, Spiral, Statheart, Wine, and Wdbc. The only exception is Liverdisorder, where GENETIC achieves a slightly lower error rate than PROPOSED. Nevertheless, this isolated case does not alter the overall pattern, which is characterized by consistent and widespread superiority of the proposed method across heterogeneous classification problems. It is particularly noteworthy that the advantage of PROPOSED is preserved even on demanding datasets such as Spiral, while especially pronounced improvements are observed on Heart, Statheart, Wine, and Wdbc relative to the competing methods.

Overall, the results support the conclusion that PROPOSED does not merely provide occasional strong outcomes, but rather exhibits greater robustness and stronger generalization ability across nearly the entire experimental setting. Therefore, it can be regarded as the most reliable method among the four compared models under the present evaluation protocol.

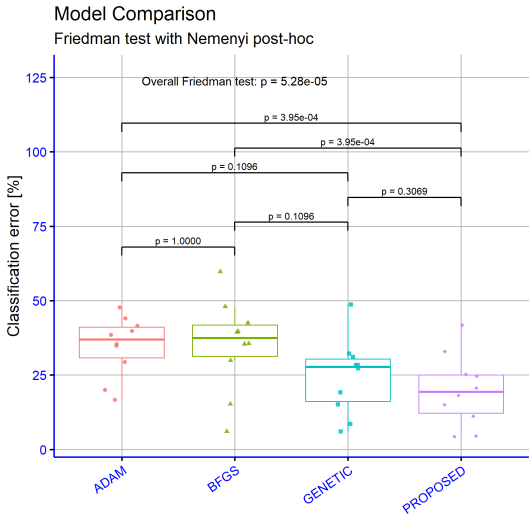

Figure 14. zzz

The statistical analysis based on the Friedman test with Nemenyi post-hoc procedure showed that there are overall statistically significant differences among the four models on the classification datasets of Table 8. Specifically, the overall Friedman p-value is 5.28×10^{-5}, which is well below 0.05, thus rejecting the null hypothesis of equivalent model performance in terms of classification error.

The pairwise comparisons reported in Figure 14 indicate that PROPOSED significantly outperforms both ADAM and BFGS, since in both cases the obtained p-value is 3.95×10^{-4}, corresponding to an extremely significant difference. In contrast, the comparisons ADAM-BFGS (p = 1.0000), ADAM-GENETIC (p = 0.1096), BFGS-GENETIC (p = 0.1096), and GENETIC-PROPOSED (p = 0.3069) are not statistically significant at the $\alpha = 0.05$ level. Therefore, the results establish PROPOSED as the statistically strongest approach relative to

the classical ADAM and BFGS schemes, whereas its superiority over GENETIC, although visible at the level of average error, is not statistically confirmed by the Nemenyi post-hoc test.

5. Discussion

The present study demonstrates that the effectiveness of ParallelBGWO does not arise from a single modification of classical BGWO, but from the systematic integration of four structural mechanisms: the partition of the total population into cooperating subpopulations, the propagation of information among clusters, the selection of an appropriate communication strategy, and the use of a flexible termination rule. This architecture substantially strengthens the balance between exploration and exploitation, which is a central requirement in large-scale optimization. The experimental evidence further suggests that parallelization is not merely used here as a technical means of reducing execution time, but as a mechanism that improves the search dynamics themselves, since the simultaneous exploration of multiple regions of the search space reduces premature convergence and facilitates the diffusion of useful information.

The analysis of the number of subpopulations confirms that the performance of the algorithm depends critically on the degree of population partitioning. The transition from a single-population structure to cooperating clusters substantially reduces computational cost; however, this relationship is not monotonically improving. The configurations with 10 and 20 clusters appear to provide the best balance, yielding the lowest or near-lowest number of function evaluations across a large portion of the benchmark suite, whereas a further increase to 30 clusters does not produce a proportional additional benefit. This finding is theoretically important because it indicates that increased parallelism does not automatically imply superior search performance. On the contrary, when the population is excessively fragmented, the internal evolutionary dynamics of each cluster are weakened, and the overall system loses part of its capacity to exploit promising regions effectively. Therefore, ParallelBGWO appears to require an operational compromise between diversity and sufficient local evolutionary pressure.

The presence of the propagation mechanism is equally decisive. The comparison between PROP = 0 and PROP = 1 shows that enabling information propagation drastically reduces the total number of objective function evaluations, which means that subpopulation cooperation is not merely an organizational feature of the algorithm but a fundamental component of its efficiency. This result suggests that locally generated high-quality information, when circulated promptly throughout the system, accelerates the displacement of the entire population toward more promising regions. This interpretation is further reinforced by the comparison of communication strategies. In both the 10- and 20-subpopulation settings, the fully bidirectional NtoN strategy consistently yields the lowest overall computational cost relative to the 1to1, 1toN, and Nto1 alternatives. This superiority is not only descriptive but also statistically supported, since the corresponding Friedman and Wilcoxon analyses indicate clear differences among the communication schemes. Consequently, the study provides convincing evidence that full information diffusion among all subpopulations is the most effective way to exploit cooperative parallelism.

A similarly clear conclusion emerges with respect to the subpopulation termination criterion. On the practical GasCycle and Tandem problems, the ANY strategy systematically produces the smallest number of function evaluations, whereas MAJORITY and especially ALL are associated with higher computational cost. From an algorithmic perspective, this result is expected, because the stricter the termination rule, the longer the system remains active in subpopulations that may already have delivered most of their useful information. However, the importance of this finding lies in its experimental confirmation that

flexibility in termination is not a minor implementation detail, but a major determinant of ParallelBGWO efficiency. In other words, the method benefits when the control framework of the search allows timely completion of the parallel cycle without imposing unnecessary waiting for all subpopulations to terminate.

The comparison with established metaheuristics substantially strengthens the position of the proposed scheme. Across the benchmark set with dimensions ranging from 25 to 150, ParallelBGWO achieves the lowest total number of function evaluations relative to GA, DE, GWO, ParallelDE, SAOP, BGWO, and WOA, while on particularly demanding functions, such as Zakharov and Sharp Ridge, the advantage is especially pronounced. This observation is critical, because it shows that the proposed framework outperforms not only the original BGWO, but also both serial and parallel competing methods. Moreover, in the practical GasCycle and Tandem experiments, ParallelBGWO maintains low values both in function evaluations and execution time as dimensionality increases, which indicates better scalability. This aspect is particularly important in large-scale optimization, because in such settings the usefulness of an algorithm is determined not only by the final solution quality, but also by the smoothness with which its computational burden grows with problem size.

The application-oriented validation through neural network training on classification datasets is also of particular significance. In that setting, ParallelBGWO attains the lowest average classification error relative to ADAM, BFGS, and Genetic, while also achieving the best performance on most of the individual datasets. This behavior indicates that the proposed framework is not restricted to artificial benchmarks, but remains operationally effective on non-convex, noisy, and practically demanding model-training problems. Even more importantly, the statistical analysis based on the Friedman and Nemenyi post-hoc procedures confirms that the superiority over ADAM and BFGS is statistically significant, whereas the advantage over Genetic, although evident at the level of average error, is not confirmed with the same degree of statistical strength. This differentiated picture enhances the credibility of the study, because it avoids overgeneralization and shows that the proposed scheme is assessed not only in terms of averages, but also under rigorous statistical criteria.

From an interpretive perspective, the results support the view that the main source of ParallelBGWO success lies in its capacity to combine effective global exploration with rapid exploitation of the collective information generated at the subpopulation level. The method appears to avoid two opposite risks that characterize many population-based optimizers: premature collapse of diversity on the one hand and excessive search dispersion without sufficient cumulative progress on the other. The fact that the best outcomes are obtained when a moderate number of clusters, enabled propagation, full NtoN communication, and the flexible ANY termination rule coexist indicates that the algorithmic architecture operates as a coherent system rather than as a mere sum of independent heuristics.

Despite this clearly positive picture, the study also leaves important room for further investigation. First, the number of subpopulations and the communication strategy remain static design choices and could in future be made adaptive according to the convergence state of the search. Second, the evaluation of the method is based primarily on function evaluations and execution time, two highly important but not exhaustive indicators. A more comprehensive assessment could incorporate a more systematic analysis of final solution quality, run-to-run variability, and behavior on even higher-dimensional or more constrained real-world problems. Third, although the neural network training experiments are particularly useful, it would be valuable to examine the performance of ParallelBGWO on deeper architectures and in settings with stronger stochastic noise.

Overall, the study provides sufficient evidence that ParallelBGWO constitutes a substantive extension of BGWO rather than a merely parallelized variant of it. The combined

experimental support from benchmark functions, practical engineering problems, and neural network training forms a coherent body of evidence in favor of the method. Accordingly, the manuscript convincingly supports the conclusion that ParallelBGWO is an efficient, stable, and adequately scalable metaheuristic framework for demanding large-scale optimization problems.

6. Conclusions

The results of the present study support the conclusion that ParallelBGWO constitutes a substantial and effective extension of the classical BGWO for high-dimensional optimization problems. The proposed architecture, which is based on the cooperation of multiple subpopulations, information propagation among them, and flexible termination rules, led to a systematic reduction in computational cost and improved convergence behavior. The experimental findings showed that the configurations with 10 and 20 subpopulations provide the most balanced performance, while the activation of the propagation mechanism decisively strengthens cluster cooperation and accelerates the transfer of useful information across the population. In addition, the NtoN communication strategy emerged as the most efficient scheme, whereas the ANY termination criterion consistently yielded the lowest computational cost relative to MAJORITY and ALL.

This superiority was not restricted to synthetic benchmark functions, but also extended to practical problems, including GasCycle and Tandem, as well as neural network training for classification tasks. Across the experimental study, ParallelBGWO exhibited better or clearly competitive behavior relative to established methods, both in terms of objective function evaluations and execution time, while in the classification application it achieved the lowest average classification error. Therefore, the study concludes that ParallelBGWO is an efficient, stable, and adequately scalable metaheuristic framework for demanding, nonlinear, and high-dimensional global optimization problems. Future research may focus on adaptive mechanisms for tuning cooperation and communication parameters, as well as on further evaluation of the method over a broader range of real-world applications.

Author Contributions: G.K., K.G.B , V.C. and I.G.T. conceived of the idea and the methodology, and G.K. and V.C. implemented the corresponding software. G.K. conducted the experiments, employing objective functions as test cases, and provided the comparative experiments. I.G.T. performed the necessary statistical tests. All authors have read and agreed to the published version of the manuscript.

Funding: This research has been financed by the European Union: Next Generation EU through the Program Greece 2.0 National Recovery and Resilience Plan, under the call RESEARCH–CREATE–INNOVATE, project name "iCREW: Intelligent small craft simulator for advanced crew training using Virtual Reality techniques" (project code: TAEDK-06195).

Institutional Review Board Statement: Not Applicable.

Informed Consent Statement: Not Applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Bertsimas, D., & Margaritis, G. (2025). Global optimization: a machine learning approach. Journal of Global Optimization, 91(1), 1-37.

2. Gauthaman, S. A., Muniasamy, A., Kamaldheen, R., Kumar, S. S. M., Murugesan, K., Viswanathan, M. K., & Devendran, H. (2026). Introduction to machine learning and optimization in healthcare. In Integrative machine learning and optimization algorithms for disease prediction (pp. 1-34). IGI Global Scientific Publishing.
3. Theodorakopoulos, L., Karras, A., & Krimpas, G. A. (2025). Optimizing apache spark MLlib: Predictive performance of large-scale models for big data analytics. Algorithms, 18(2), 74
4. Nassef, A. M., Abdelkareem, M. A., Maghrabie, H. M., & Baroutaji, A. (2023). Review of metaheuristic optimization algorithms for power systems problems. Sustainability, 15(12), 9434.
5. Recalde, A., Cajo, R., Velasquez, W., & Alvarez-Alvarado, M. S. (2024). Machine learning and optimization in energy management systems for plug-in hybrid electric vehicles: A comprehensive review. Energies, 17(13), 3059.
6. Kyrou, G., Charilogis, V., & Tsoulos, I. G. (2026). A Novel Method That Is Based on Differential Evolution Suitable for Large-Scale Optimization Problems. Foundations, 6(1), 2.
7. Tang, K.; Li, X.; Suganthan, P.N.; Yang, Z.; Weise, T. Benchmark Functions for the CEC'2010 Special Session and Competition on Large-Scale Global Optimization; Nature Inspired Computation and Applications Laboratory, USTC: Hefei, China, 2007; Volume 24, pp. 1–18.
8. Li, X.; Tang, K.; Omidvar, M.N.; Yang, Z.; Qin, K. Benchmark functions for the CEC 2013 special session and competition on large-scale global optimization. Gene 2013, 7, 8
9. Molina, D.; Herrera, F. Iterative hybridization of DE with local search for the CEC'2015 special session on large scale global optimization. In 2015 IEEE Congress on Evolutionary Computation (CEC); IEEE: Piscataway, NJ, USA, 2015; pp. 1974–1978. 11. Li, P.; Hao, J.; Tang, H.; Fu, X.; Zhen, Y.; Tang, K. Bridging evolutionary algorithms and reinforcement learning: A comprehensive survey on hybrid algorithms. IEEE Trans. Evol. Comput. 2024, 29, 1707–1728.
10. Long, W., Jiao, J., Liang, X., & Tang, M. (2018). Inspired grey wolf optimizer for solving large-scale function optimization problems. Applied Mathematical Modelling, 60, 112-126.
11. Long, W., Cai, S., Jiao, J., & Tang, M. (2020). An efficient and robust grey wolf optimizer algorithm for large-scale numerical optimization: W. Long et al. Soft Computing, 24(2), 997-1026.
12. Powell, M.J.D. A tolerant algorithm for linearly constrained optimization calculations. Math. Program. 1989, 45, 547–566.
13. Charilogis, V., & Tsoulos, I. G. (2022). Toward an ideal particle swarm optimizer for multidimensional functions. Information, 13(5), 217.
14. Wan, H. (2026). Applying the genetic algorithm to optimization problems. WIT Transactions on Information and Communication Technologies, 2.
15. Pereira, C. M. N. A., Schirru, R., & Martinez, A. S. (2026). Learning an optimized classification system from a data base of time series patterns using genetic algorithms. WIT Transactions on Information and Communication Technologies, 22.
16. Koyuncuoğlu, M. U. (2026). Sustainable configuration of production lines: Multi-objective energy-efficient optimization with adaptive differential evolution and golden ratio algorithms. Engineering Applications of Artificial Intelligence, 165, 113469.
17. Horrillo-Quintero, P., Garcia-Trivino, P., Carrasco-Gonzalez, D., & Fernandez-Ramirez, L. M. (2026). Optimal control strategy based on differential evolution algorithm for seamless transition between islanded and grid-connected operation modes in microgrid clusters. Expert Systems with Applications, 297, 129461.
18. Gupta, S., & Gautam, K. (2026). Enhanced cooperative learning and local search integrated particle swarm optimization for global optimization and coverage enhancement in wireless sensor network. Swarm and Evolutionary Computation, 100, 102262.
19. Li, Z., Wang, P., Feng, Y., Zhang, G., Guo, Z., Zhang, M., & Zhang, Y. (2026). Optimization of manipulator inverse kinematics using improved particle swarm algorithm for enhanced manufacturing efficiency. Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture, 240(1-2), 188-202.
20. Chen, F., Wu, X., Wang, Z., Qi, W., & Li, P. (2026). A New Ant Colony Optimization-Based Dynamic Path Planning and Energy Optimization Model in Wireless Sensor Networks for Mobile Sink by Using Mixed-Integer Linear Programming. Biomimetics, 11(1), 44.
21. Yang, W., Huang, C., & Ji, P. (2026). Ant colony algorithm optimized by local search and adaptive strategy for drilling sequence path planning. Journal of Mechanical Science and Technology, 1-11
22. Saini, G., Jadon, S. S., & Chaube, S. (2026). Learning-aided Artificial Bee Colony with neural knowledge transfer for global optimization. Scientific Reports.
23. Saini, G., & Jadon, S. S. (2026). A neural knowledge learning-driven artificial bee colony algorithm with reinforcement adaptation for global optimization. Applied Soft Computing, 114662.
24. Nouri, P., Kamarposhti, M. A., Nouri, T., & Radmehr, M. (2026). Optimization of fuzzy logic controller in the converter of a standalone solar power system using the firefly algorithm. Scientific Reports.
25. Sutrisno, D., Ardhika, D. Y., Siddiq, S. A., Prakasa, M. A., Fauzi, M. A., Widiyawati, E., ... & Robandi, I. (2026). Application of Firefly Algorithm for Optimizing the Available Transfer Capability in the Sulbagsel Electricity System of Indonesia. International Journal of Intelligent Engineering & Systems, 19(3).

26. Senandes, R. S., Brante, G., Souza, R. D., Azarbahram, A., & López, O. L. A. (2026). Bat Algorithm-Based Energy Beamforming for Wireless Power Transfer with Dynamic Metasurface Antennas. IEEE Transactions on Communications.
27. Alsumaiei, A. A. (2026). Physics-constrained neural network for daily pan evaporation forecasting in hyper-arid climates optimized by Bat Algorithm. Journal of Hydrology, 134936.
28. Almufti, S. M., Hassan, N. S., Mercado, M. C., Felix, J. W. G., Barbosa, T. S., & Supan, F. E. (2026). Survey on whale optimization algorithm: from fundamental principles to modern adaptations and applications.
29. Su, Y., & Liu, Y. (2026). A novel marine predator whale optimization algorithm for global numerical optimization. Engineering Optimization, 58(1), 203-239.
30. Alauthman, M., Al-Qerem, A., Almomani, A., Ishtaiwi, A. M., Aldweesh, A., Arafah, M., ... & Gupta, B. B. (2026). Enhancing Web of Things Security Using Harris Hawks Optimization with Reinforcement Learning. International Journal of Cognitive Computing in Engineering.
31. Zeng, J., Chen, M. R., Guo, Z., Zhang, Z., & Zhong, S. (2025, October). An Improved Large-Scale Multi-objective Competitive Swarm Optimizer Based on Harris Hawks Optimization. In International Conference on Behavioural and Social Computing (pp. 30-42). Singapore: Springer Nature Singapore.
32. Varshney, M., & Ali, M. (2026). A Modified Aquila Optimizer for Application to Plate–Fin Heat Exchangers Design Problem. Mathematics, 14(3), 431.
33. Wreford, A. I. OPTIMIZATION OF DEEP NEURAL NETWORKS FOR HEART DISEASE DIAGNOSIS USING THE AQUILA OPTIMIZER: BRIDGING AI AND BIO-INSPIRED COMPUTATION. Journal of Science and Technology, 4(1), 62-70.
34. Sheng, W., Yang, X., Jia, D., Liu, K., Han, Q., & Li, C. (2026). Fault Location Method for Distribution Networks Based on Cluster Partitioning and Arithmetic Optimization Algorithm. Processes, 14(3), 493.
35. Toğan, V., Said Sulub, A., Azim Eirgash, M., & Mostofi, F. (2026). Project Scheduling to Minimize the Time and Cost in Large-Scale Construction Projects with Repulsion-Based Improved Arithmetic Optimization. Journal of Construction Engineering and Management, 152(3), 04025277.
36. Kyrou, G., Charilogis, V., & Tsoulos, I. G. (2025). A Novel Magnificent Frigatebird Optimization Algorithm with Proposed Movement Strategies for Enhanced Global Search. Analytics, 5(1), 1.
37. Lin, J., Zhang, X., Li, R., Huang, Y., & Qin, Q. (2026). Multi-Strategy Enhanced Hybrid Salp Swarm Algorithm for Robust Association Rule Mining in Dynamic Environments. International Journal of Pattern Recognition and Artificial Intelligence.
38. Nguyen, K. D., Tran, T. T., & Vo, D. N. (2026). An efficient Salp Swarm Algorithm for optimizing wind farm layouts with neighboring farm impacts. International Journal of Green Energy, 23(2), 420-435.
39. AMUSAT, R. O., OCHENI, U. U., SHODIYA, S., & SALAU, R. (2026). Enhanced MPPT Using Hybrid Smell Agent and Particle Swarm Optimization under Partial Shading in PV Arrays. UNIABUJA Journal of Engineering and Technology (UJET), 3(1), 71-81.
40. Kyrou, G., Tsoulos, I. G., Gianni, A. M., & Charilogis, V. (2025). Parallel smell agent optimization (SAO): collaborative subpopulations for accelerated convergence. Symmetry, 17(4), 592.
41. Chen, J., Zhu, H., Shu, T., Cao, C., Deng, Y., & Cheng, Q. (2026). Balanced Grey Wolf Optimizer Algorithm for Backpropagation Neural Networks. Mathematics, 14(3), 554
42. Adhikary, J., & Acharyya, S. (2022). Randomized Balanced Grey Wolf Optimizer (RBGWO) for solving real life optimization problems. Applied Soft Computing, 117, 108429.
43. Wang, J., Lin, D., Zhang, Y., & Huang, S. (2022). An adaptively balanced grey wolf optimization algorithm for feature selection on high-dimensional classification. Engineering Applications of Artificial Intelligence, 114, 105088.
44. Ali, M.M.; Kaelo, P. Improved particle swarm algorithms for global optimization. Appl. Math. Comput. 2008, 196, 578–593
45. Koyuncu, H.; Ceylan, R. A PSO based approach: Scout particle swarm algorithm for continuous global optimization problems. J. Comput. Des. Eng. 2019, 6, 129–142.
46. Siarry, P.; Berthiau, G.; Durdin, F.; Haussy, J. Enhanced simulated annealing for globally minimizing functions of many-continuous variables. ACM Trans. Math. Softw. (TOMS) 1997, 23, 209–228.
47. LaTorre, A.; Molina, D.; Osaba, E.; Poyatos, J.; Del Ser, J.; Herrera, F. A prescription of methodological guidelines for comparing bio-inspired optimization algorithms. Swarm Evol. Comput. 2021, 67, 100973.
48. Tsoulos, I.G.; Charilogis, V.; Kyrou, G.; Stavrou, V.N.; Tzallas, A. OPTIMUS: A Multidimensional Global Optimization Package. J. Open Source Softw. 2025, 10, 7584.
49. Luo, B., Su, X., Zhang, S., Yan, P., Liu, J., & Li, R. (2025). Analysis of a novel gas cycle cooler with large temperature glide for space cooling. Energy, 326, 136294.
50. Keerthika, R., Niranjan, S. P., & Komala Durga, B. (2025). A Survey on the tandem queueing models. Scope, 14, 134-148.
51. Kelly, M., Longjohn, R., & Nottingham, K. (2023, November). The UCI machine learning repository.
52. Derrac, J., Garcia, S., Sanchez, L., & Herrera, F. (2015). Keel data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework. J. Mult. Valued Logic Soft Comput, 17, 255-287.

53. Abiodun, O. I., Jantan, A., Omolara, A. E., Dada, K. V., Mohamed, N. A., & Arshad, H. (2018). State-of-the-art in artificial neural network applications: A survey. Heliyon, 4(11).
54. Suryadevara, S., & Yanamala, A. K. Y. (2021). A Comprehensive Overview of Artificial Neural Networks: Evolution, Architectures, and Applications. Revista de Inteligencia Artificial en Medicina, 12(1), 51-76.
55. Kononenko, I., Šimec, E., & Robnik-Šikonja, M. (1997). Overcoming the myopia of inductive learning algorithms with RELIEFF. Applied Intelligence, 7(1), 39-55.
56. Perantonis, S. J., & Virvilis, V. (1999). Input feature extraction for multilayered perceptrons using supervised principal component analysis. Neural processing letters, 10(3), 243-252.
57. S. J. Perantonis, V. Virvilis, Input Feature Extraction for Multilayered Perceptrons Using Supervised Principal Component Analysis, Neural Processing Letters **10**, pp 243–252, 1999.
58. Garcke, J., & Griebel, M. (2002). Classification with sparse grids using simplicial basis functions. Intelligent data analysis, 6(6), 483-502.
59. Cestnik, B., Kononenko, I., & Bratko, I. (1987, May). ASSISTANT 86: A knowledge-elicitation tool for sophisticated users. In Proceedings of the 2nd European conference on European working session on learning (pp. 31-45).
60. Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988, November). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In Proceedings of the annual symposium on computer application in medical care (p. 261).
61. Wolberg, W. H., & Mangasarian, O. L. (1990). Multisurface method of pattern separation for medical diagnosis applied to breast cytology. Proceedings of the national academy of sciences, 87(23), 9193-9196.
62. Raymer, M. L., Doom, T. E., Kuhn, L. A., & Punch, W. F. (2003). Knowledge discovery in medical and biological datasets using a hybrid Bayes classifier/evolutionary algorithm. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 33(5), 802-813.
63. Zhong, P., & Fukushima, M. (2007). Regularized nonsmooth Newton method for multi-class support vector machines. Optimisation Methods and Software, 22(1), 225-236.
64. D. P. Kingma, J. L. Ba, ADAM: a method for stochastic optimization, in: Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015), pp. 1–15, 2015.
65. Xia, J. H., & Kumta, A. S. (2010). Feedforward neural network trained by BFGS algorithm for modeling plasma etching of silicon carbide. IEEE transactions on plasma science, 38(2), 142-148.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.